Lecture 6b: Reductions Recap

Alessandro Chiesa

EPFL School of Computer and Communication Sciences

Lecture 6 Recap

Reductions

Informally

Reducibility Use knowledge about complexity of one language to reason about the complexity of other in an easy way

Reduction Way to show that to solve one problem (A), it is sufficient to solve another problem (B)

A reduces to B...

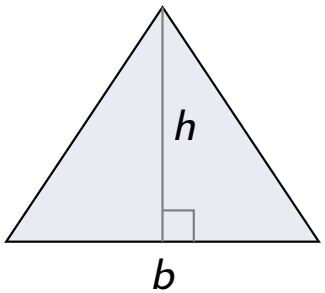
- Many examples (e.g. calculate area of rectangle reduces to calculate its width and height)
- Reductions quantify relative hardness of problems
  - If problem B is easy then problem A is easy too
  - If problem A is hard then problem B is hard too

A reduction f from triangle to rectangle area

A reduction f from triangle to rectangle area

Triangle-Area $=$

$\{\langle h, b, t\rangle \in \{0,1\}^* \mid h * b/2 \geq t\}$

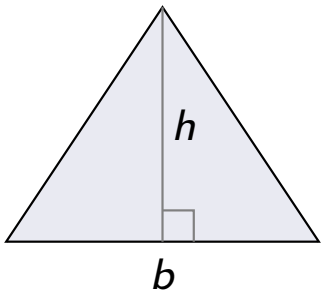


In language if area is at least t

A reduction f from triangle to rectangle area

Triangle-Area $=$

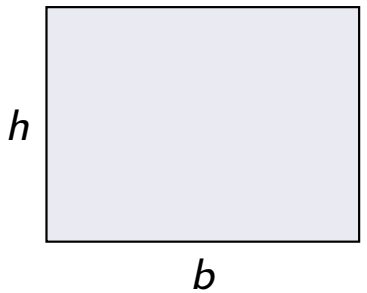
$\{\langle h, b, t\rangle \in \{0,1\}^* \mid h * b/2 \geq t\}$



In language if area is at least t

Rectangle-Area $=$

$\{\langle h, b, r\rangle \in \{0,1\}^* \mid h * b \geq r\}$

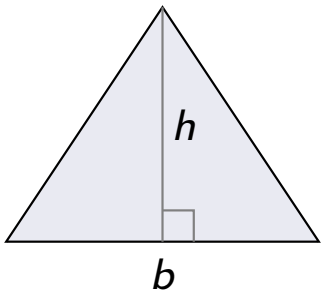


In language if area is at least r

A reduction f from triangle to rectangle area

Triangle-Area $=$

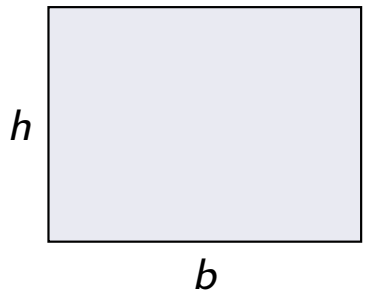
$\{\langle h, b, t\rangle \in \{0,1\}^* \mid h * b/2 \geq t\}$



In language if area is at least t

Rectangle-Area $=$

$\{\langle h, b, r\rangle \in \{0,1\}^* \mid h * b \geq r\}$



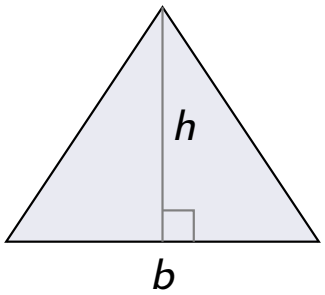
In language if area is at least r

Suppose we have a Turing machine TM_{RA} for deciding Rectangle-Area. How can we use it form a decider TM_{TA} for Triangle-Area?

A reduction f from triangle to rectangle area

Triangle-Area $=$

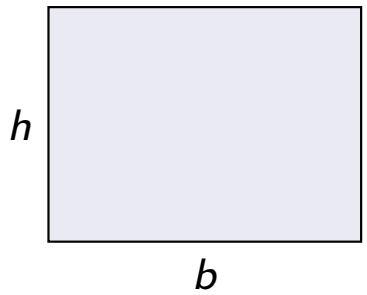
$\{\langle h, b, t\rangle \in \{0,1\}^* \mid h * b/2 \geq t\}$



In language if area is at least t

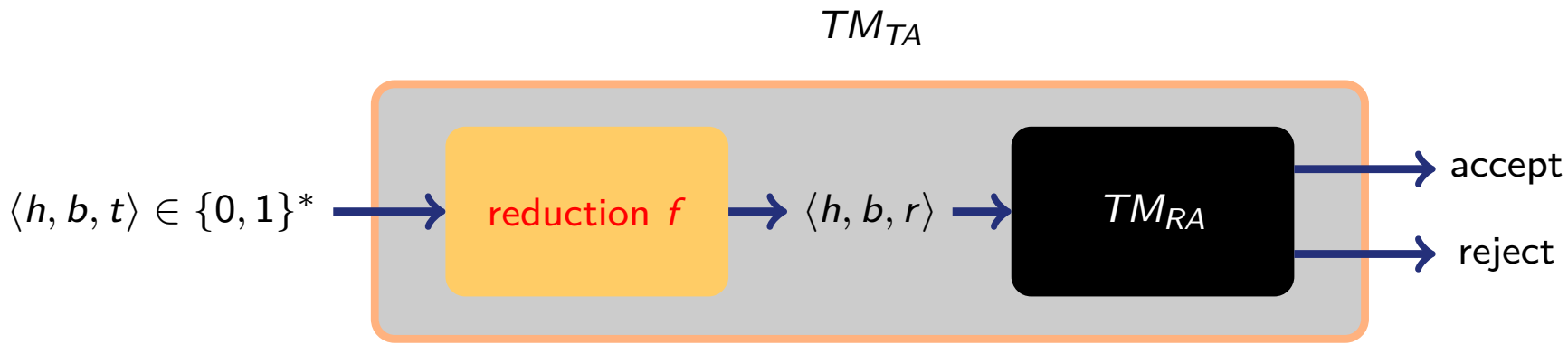
Rectangle-Area $=$

$\{\langle h, b, r\rangle \in \{0,1\}^* \mid h * b \geq r\}$

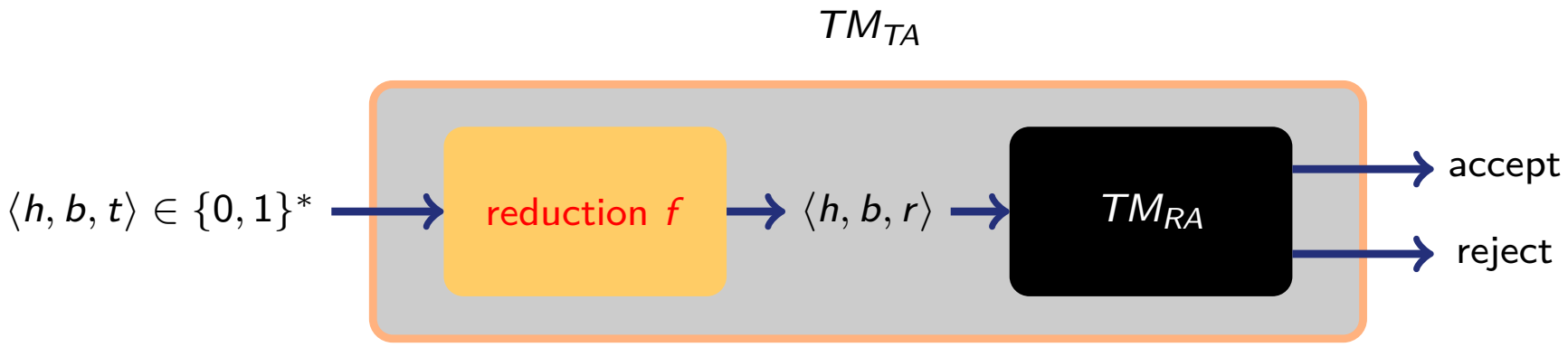


In language if area is at least r

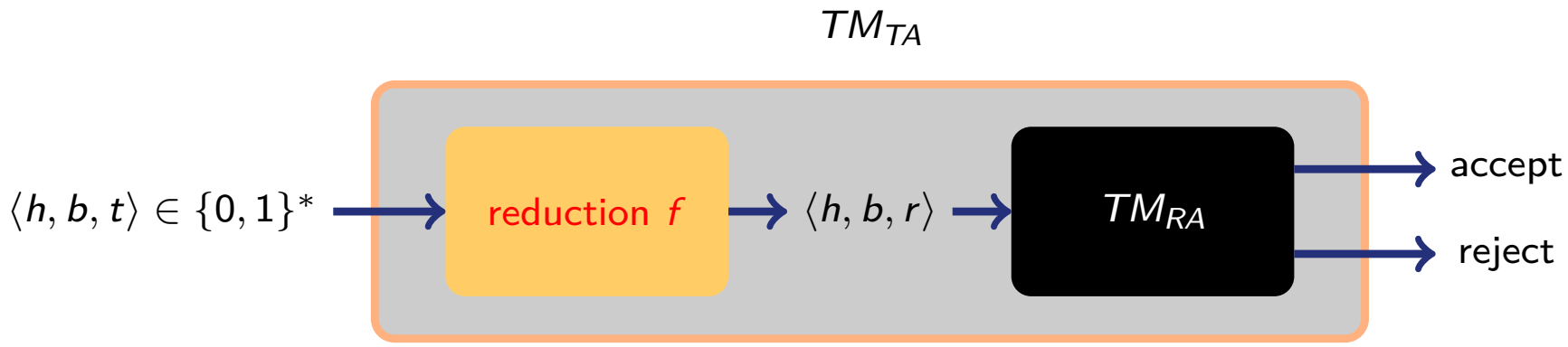
Suppose we have a Turing machine TM_{RA} for deciding Rectangle-Area. How can we use it form a decider TM_{TA} for Triangle-Area?



A reduction f from triangle to rectangle area

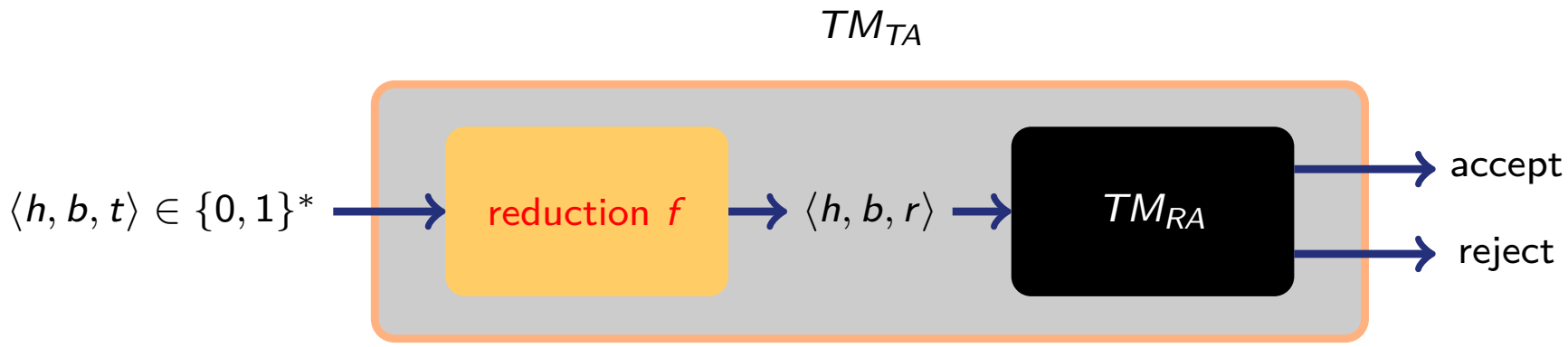


A reduction f from triangle to rectangle area

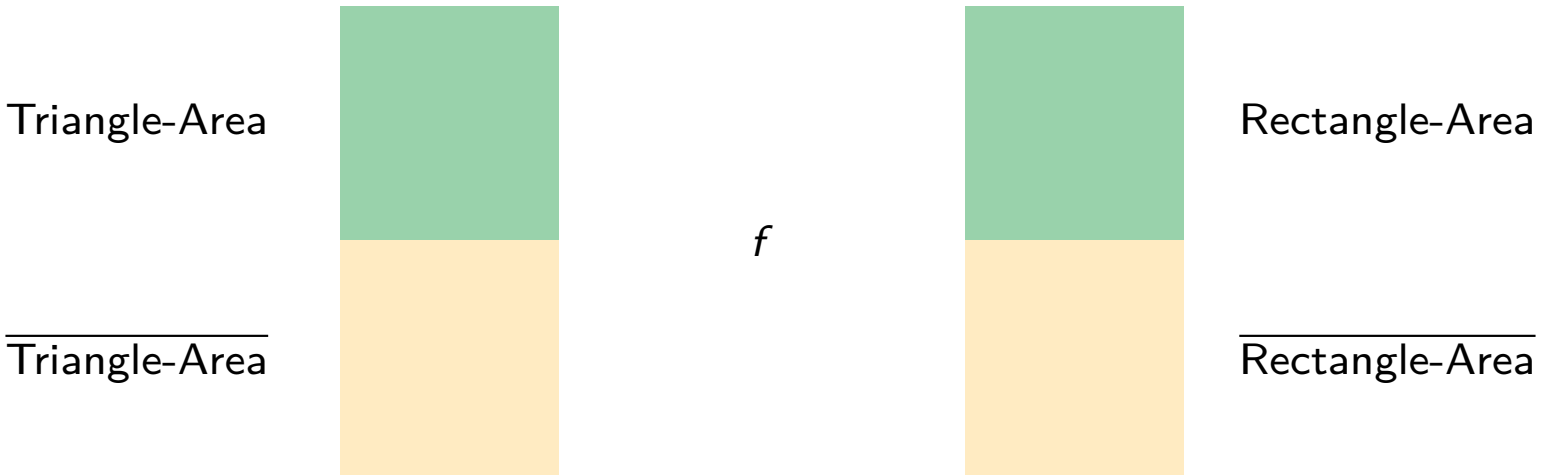


Define $f(\langle h, b, t \rangle) = \langle h, b, 2t \rangle$. Then

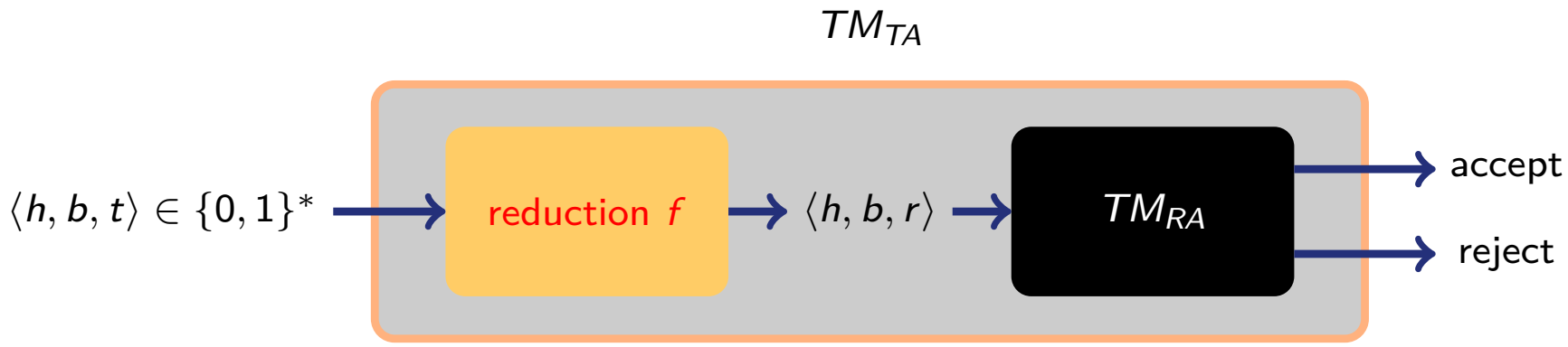
A reduction f from triangle to rectangle area



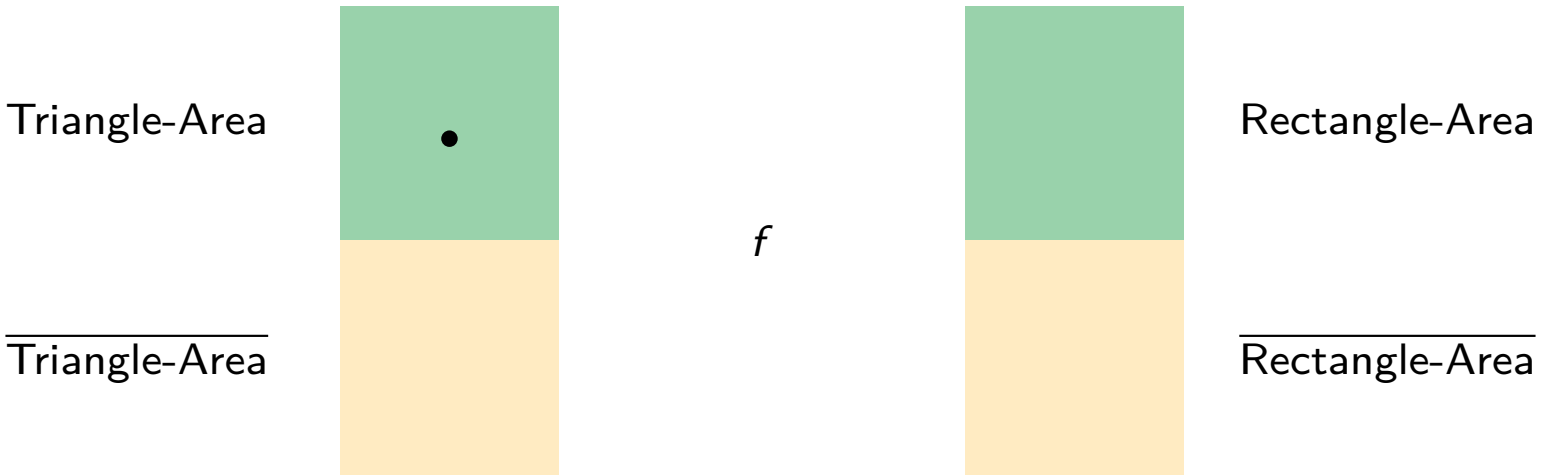
Define $f(\langle h, b, t\rangle) = \langle h, b, 2t\rangle$. Then



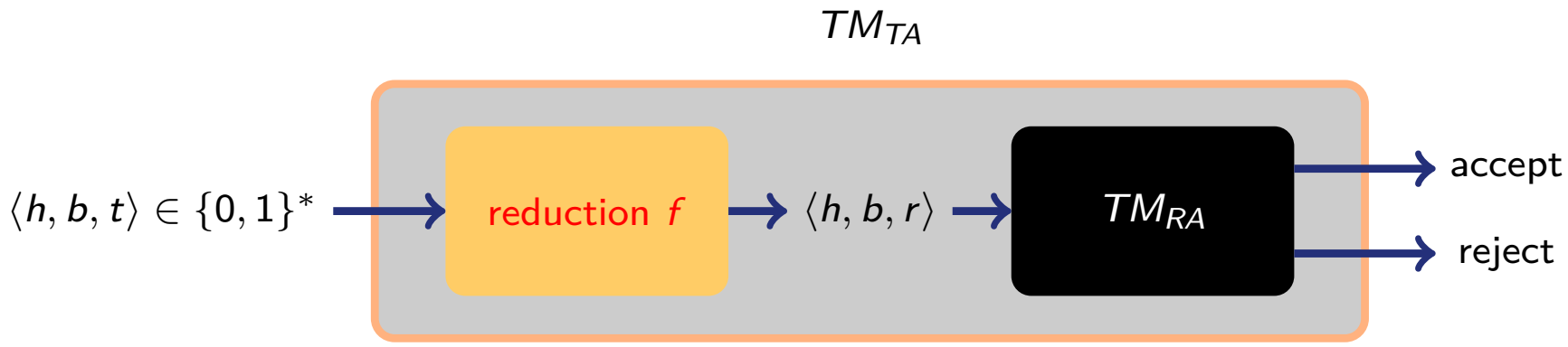
A reduction f from triangle to rectangle area



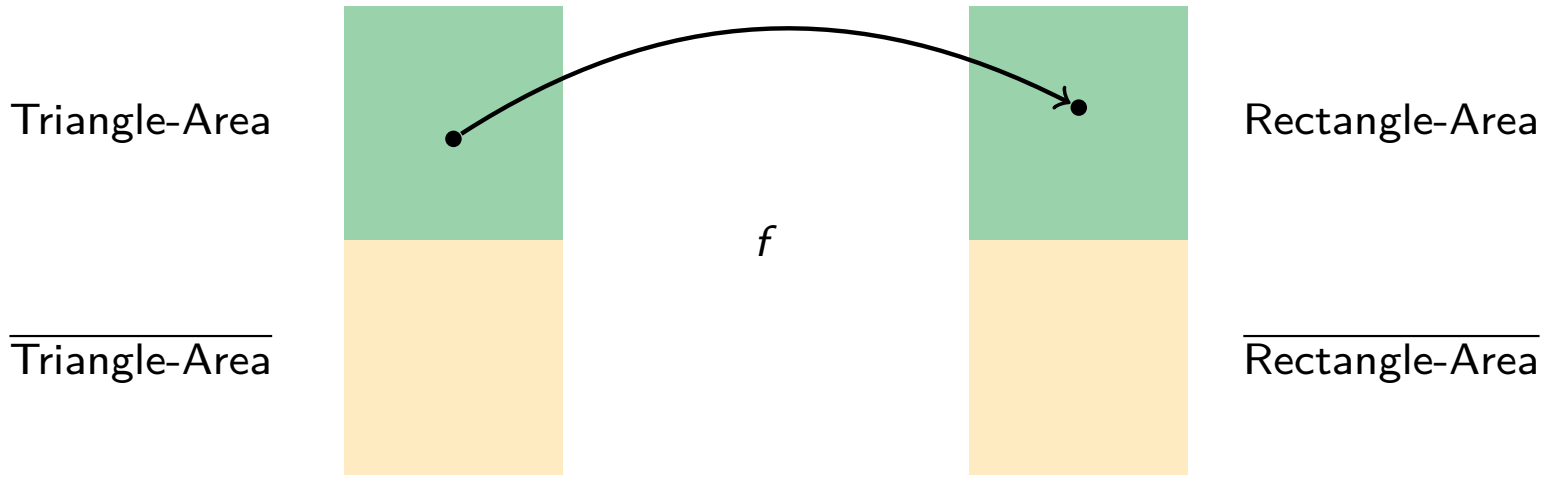
Define $f(\langle h, b, t\rangle) = \langle h, b, 2t\rangle$. Then



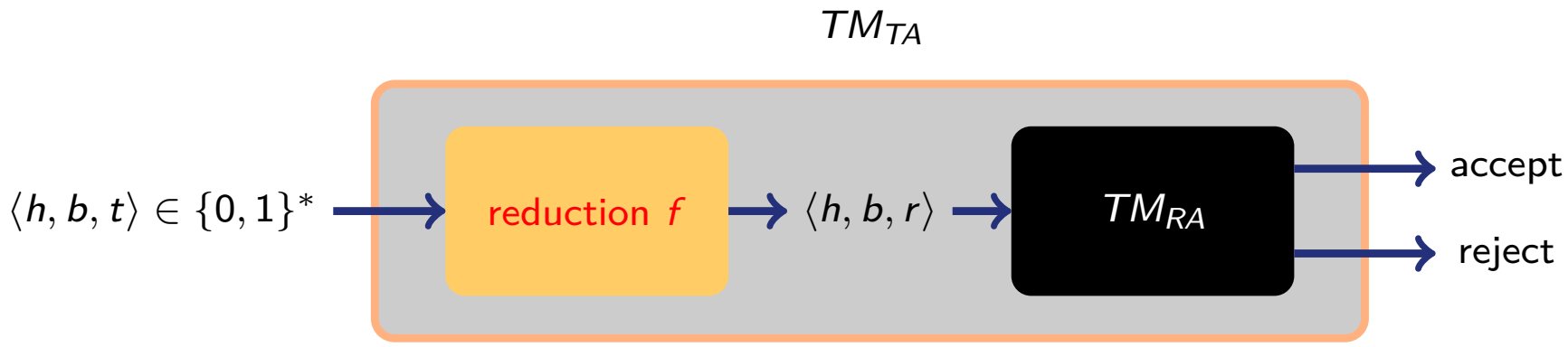
A reduction f from triangle to rectangle area



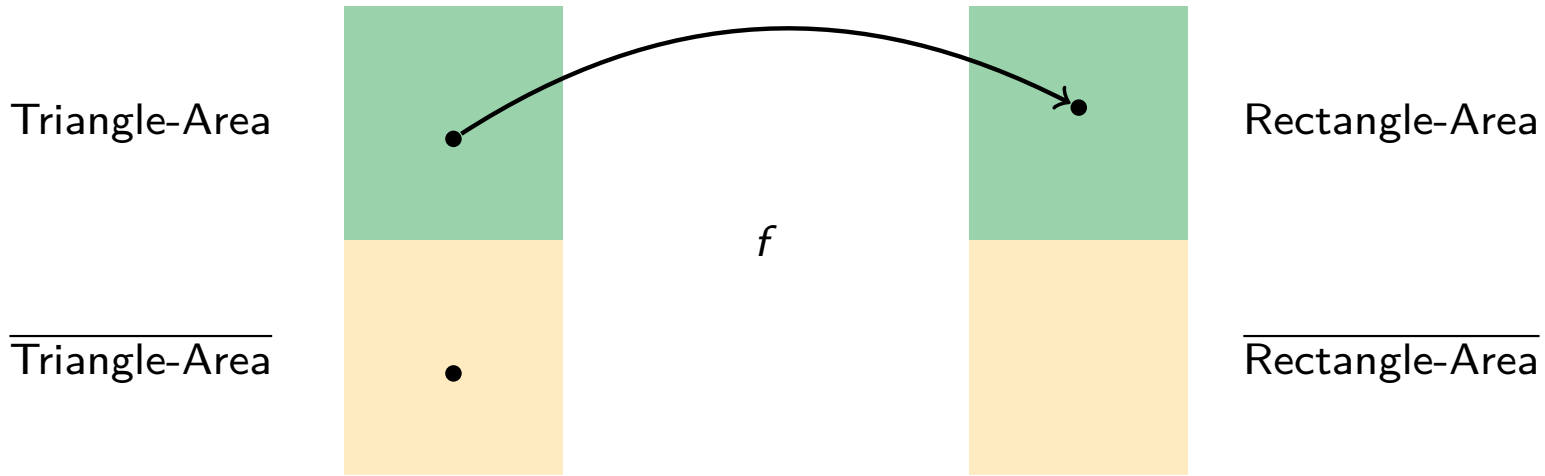
Define $f(\langle h, b, t\rangle) = \langle h, b, 2t\rangle$. Then



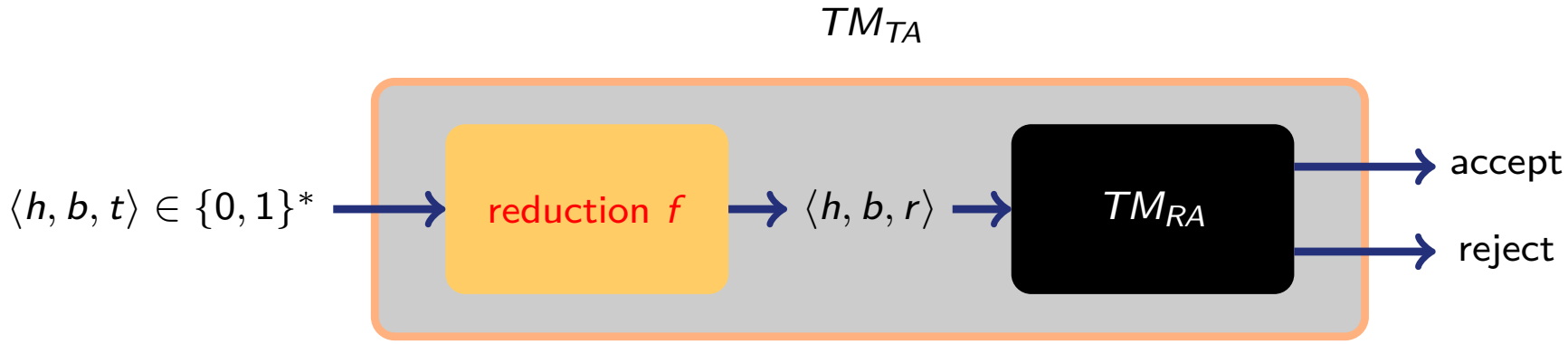
A reduction f from triangle to rectangle area



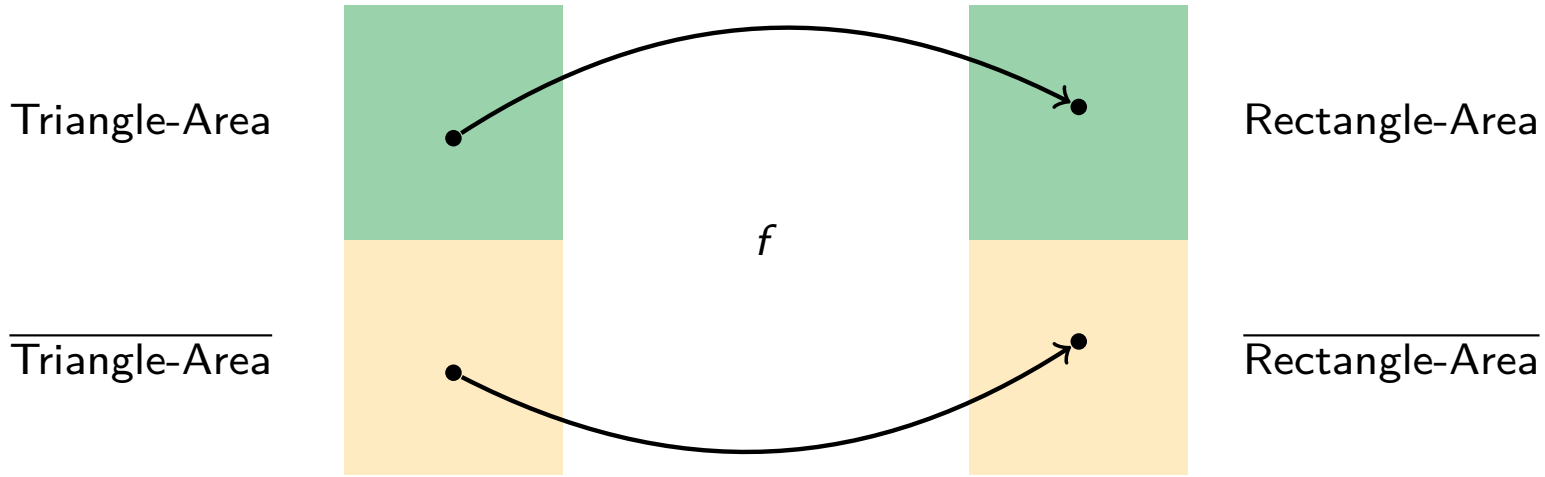
Define $f(\langle h, b, t\rangle) = \langle h, b, 2t\rangle$. Then



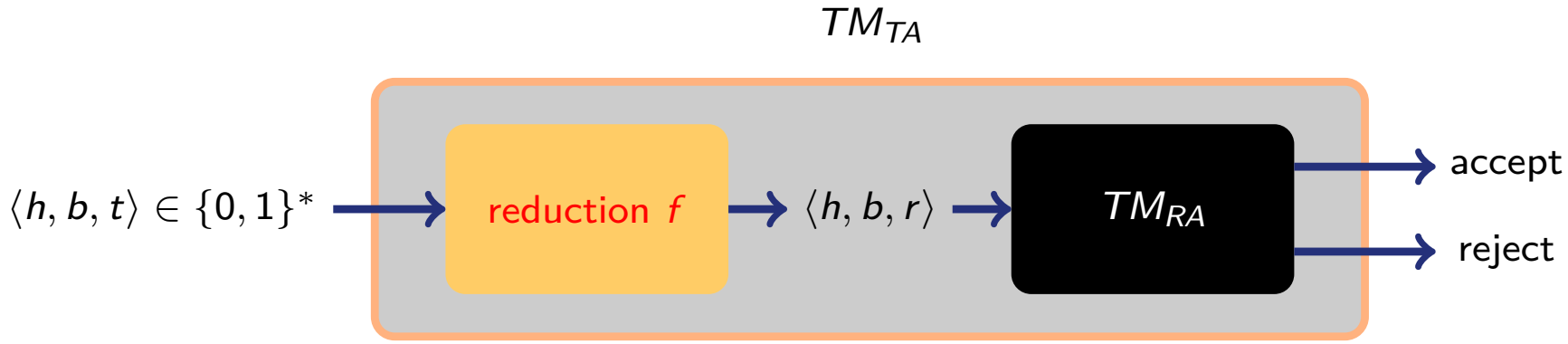
A reduction f from triangle to rectangle area



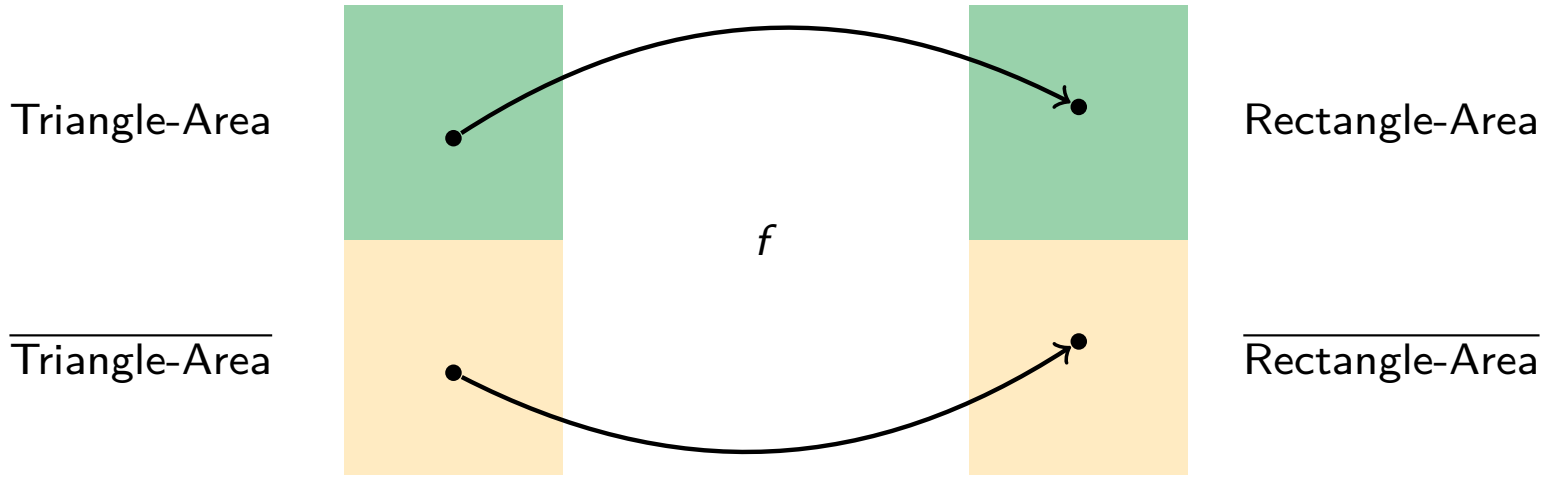
Define $f(\langle h, b, t\rangle) = \langle h, b, 2t\rangle$. Then



A reduction f from triangle to rectangle area

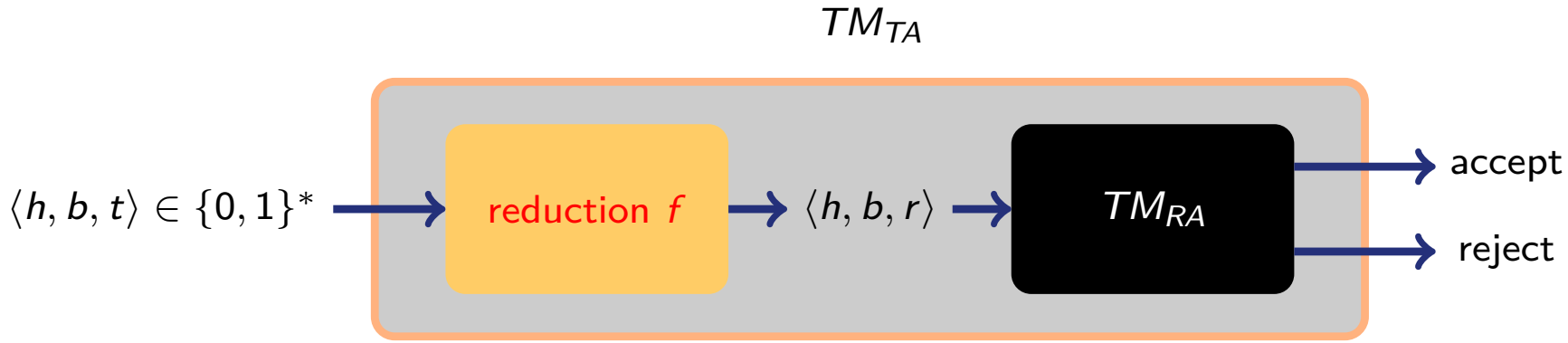


Define $f(\langle h, b, t\rangle) = \langle h, b, 2t\rangle$. Then

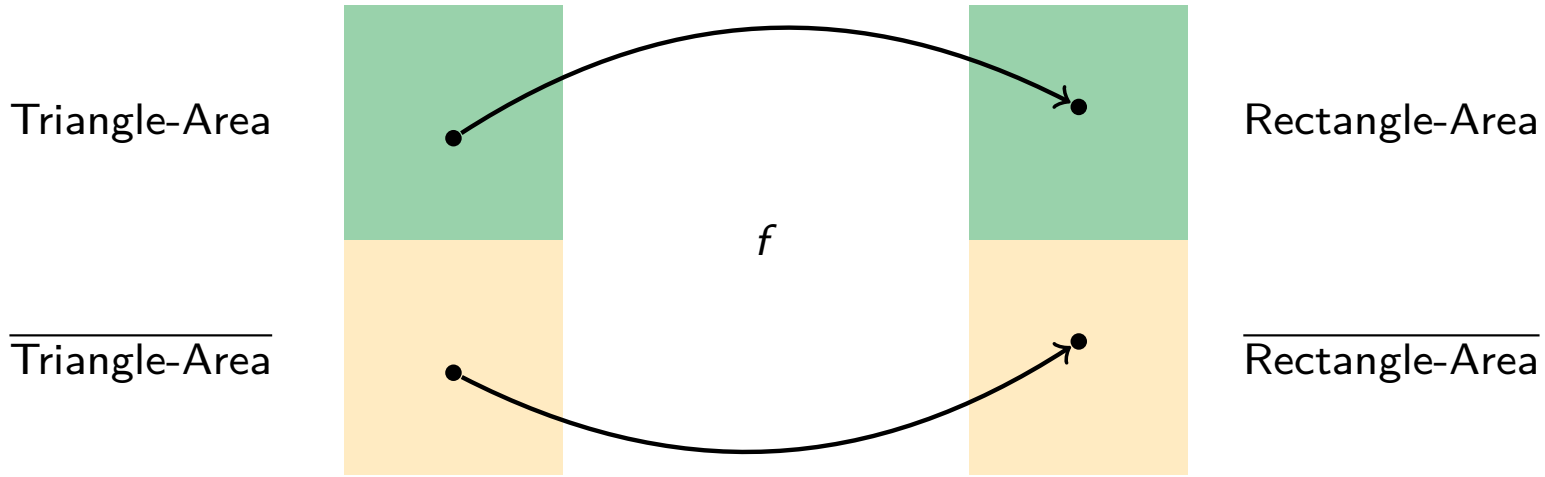


- f does not use the knowledge whether input is in language or not

A reduction f from triangle to rectangle area



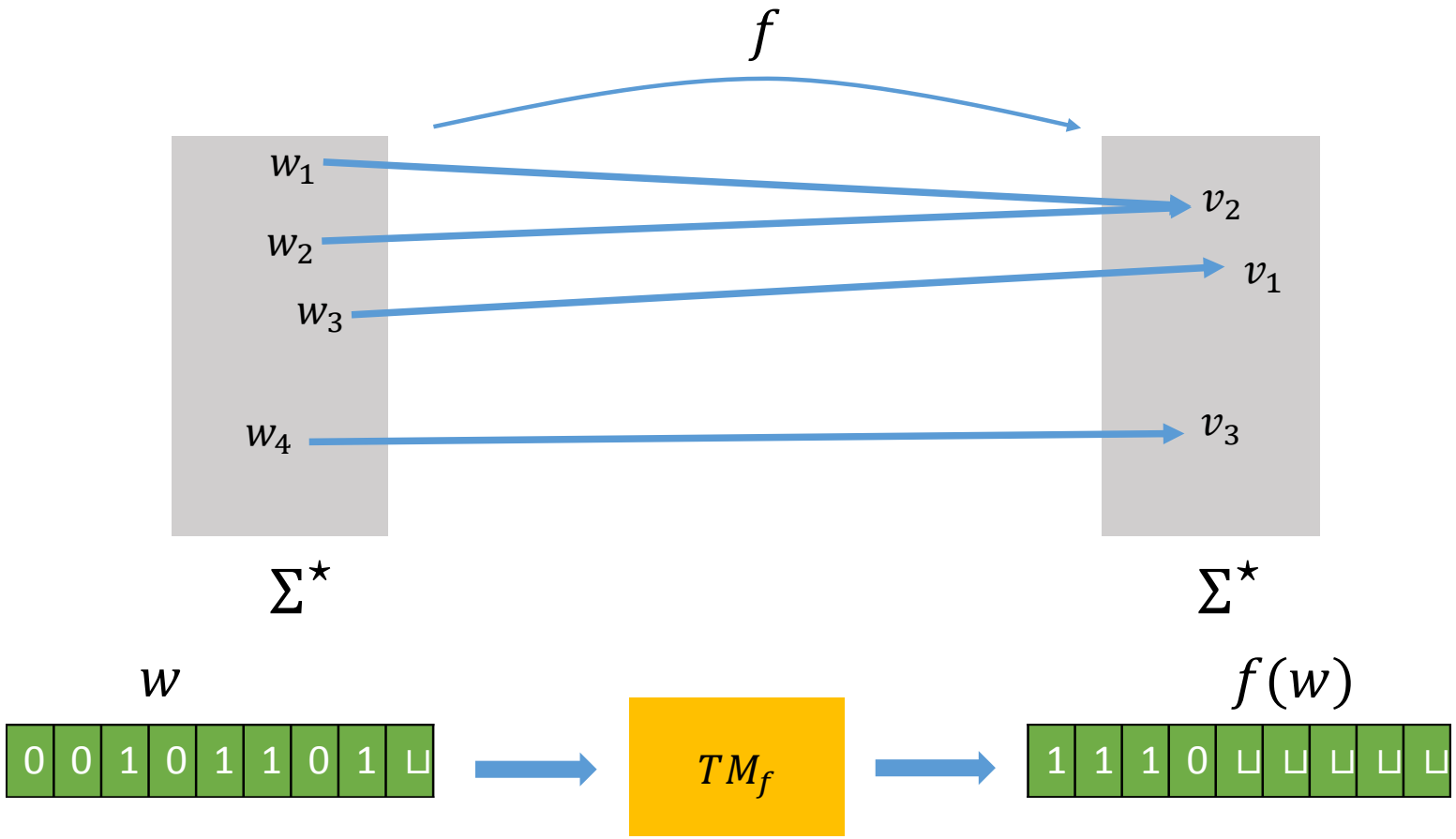
Define $f(\langle h, b, t\rangle) = \langle h, b, 2t\rangle$. Then



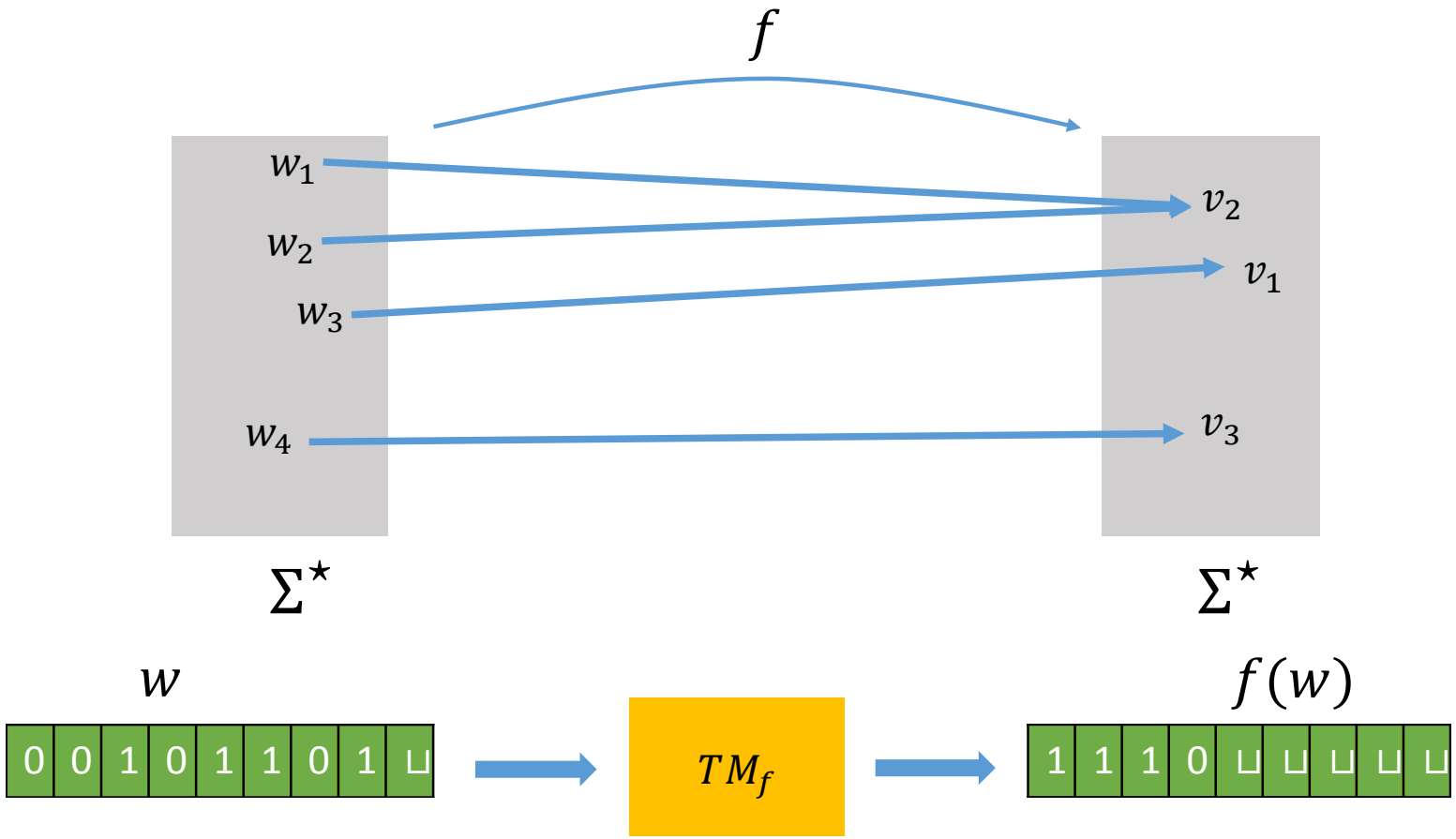
- f does not use the knowledge whether input is in language or not
- f can be computed with a TM

Mapping Reductions

Reductions Part 1: Computability

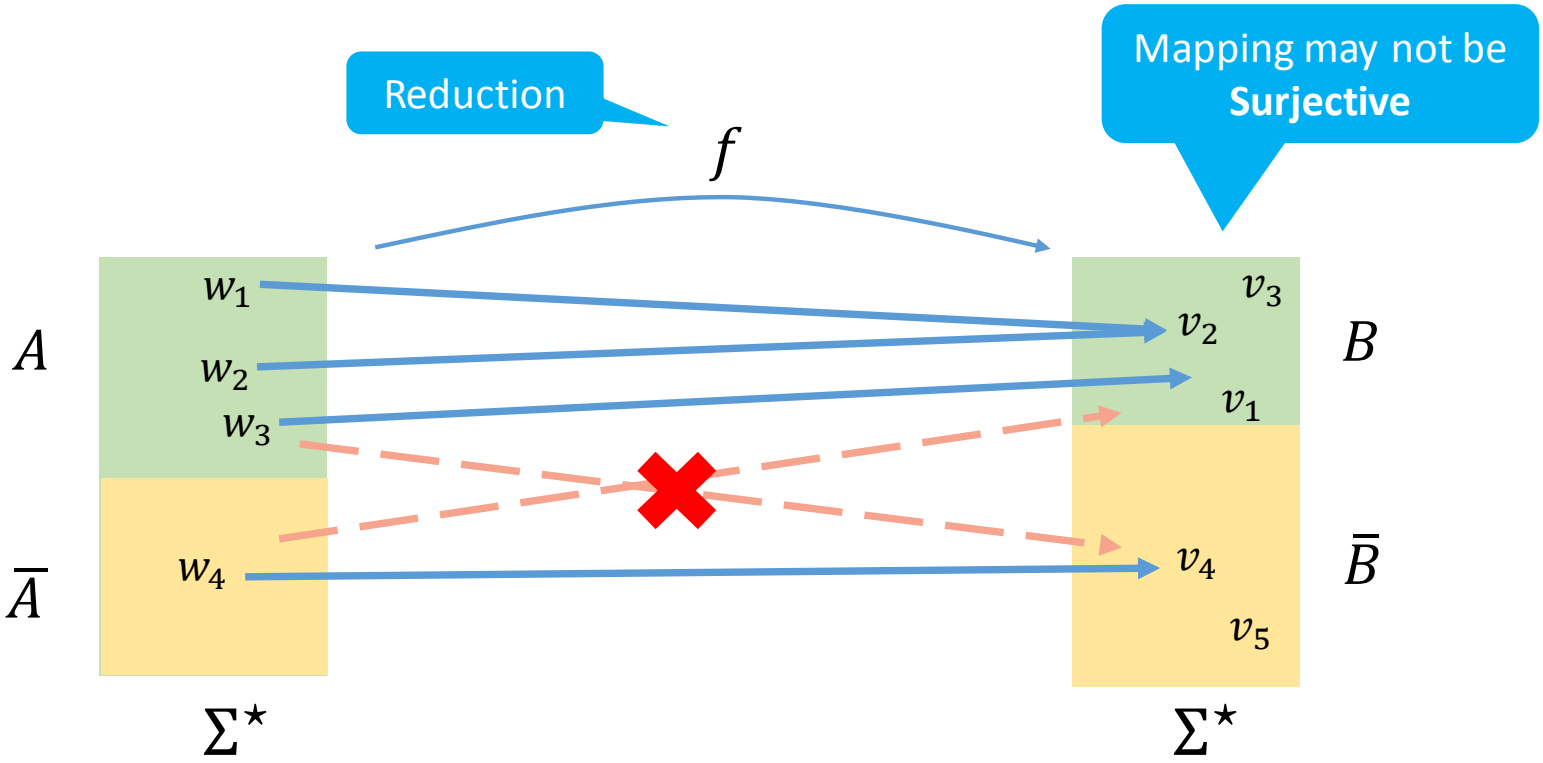


Reductions Part 1: Computability

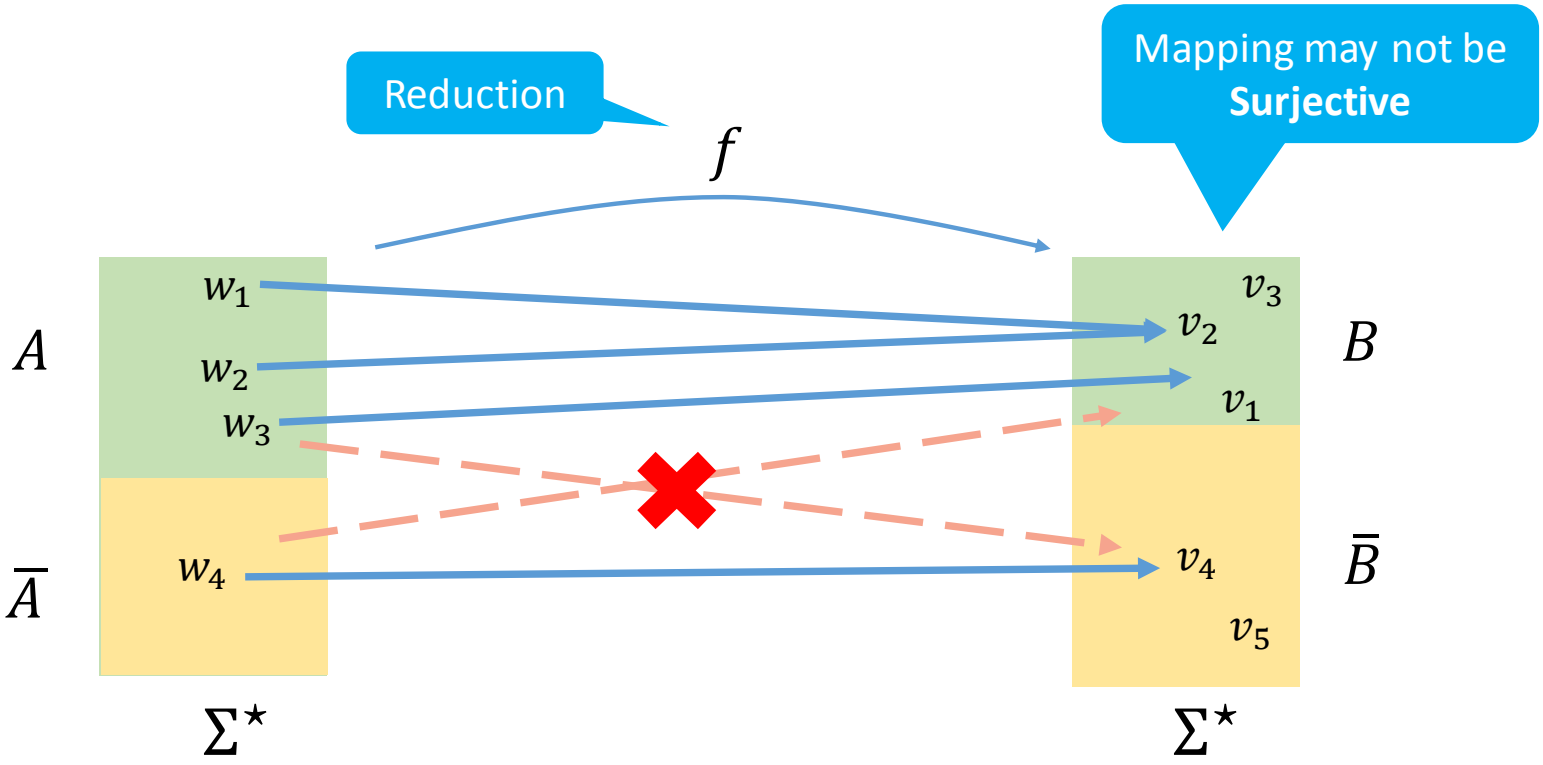


Definition: A function $f : \Sigma^* \to \Sigma^*$ is a *computable function* if some TM M, on every input w halts with just $f(w)$ on its tape

Reductions Part 2: Correctness



Reductions Part 2: Correctness



Definition: Language A is **mapping reducible** to language B, written $A \leq_m B$, if there is a computable function $f : \Sigma^* \to \Sigma^*$ such that for every $w \in \Sigma^*$:

$$w \in A \Leftrightarrow f(w) \in B$$

Theorem: If $A \leq_m B$ and B is decidable, then A is decidable

Theorem: If $A \leq_m B$ and B is decidable, then A is decidable

Proof:

Theorem: If $A \leq_m B$ and B is decidable, then A is decidable

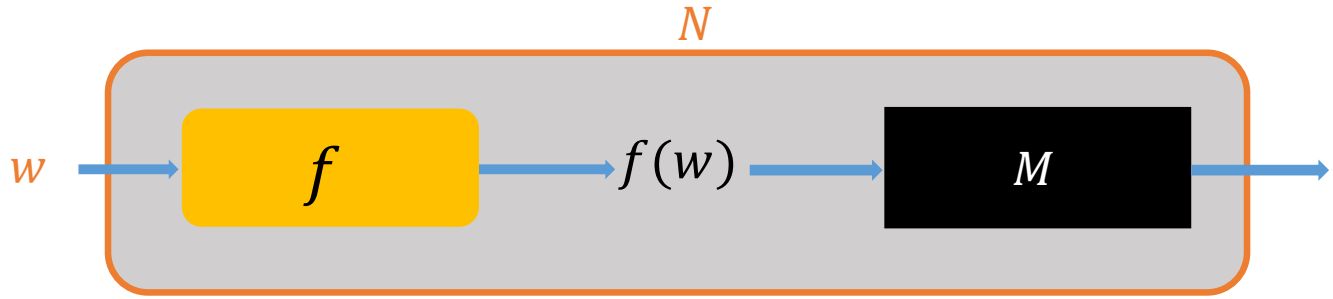
Proof:

- Assume that M is a decided for B and f is a reduction from A to B

Theorem: If $A \leq_m B$ and B is decidable, then A is decidable

Proof:

- Assume that M is a decided for B and f is a reduction from A to B
- Let N be a TM as follows:

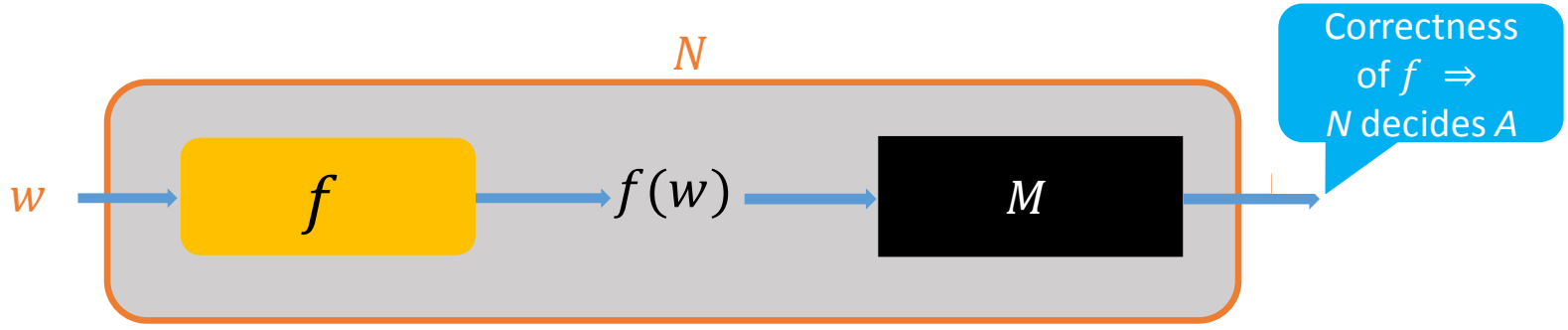


- N = "On input w :
  1. Compute $f(w)$
  2. Run M on input $f(w)$ and output whatever M outputs"

Theorem: If $A \leq_m B$ and B is decidable, then A is decidable

Proof:

- Assume that M is a decided for B and f is a reduction from A to B
- Let N be a TM as follows:



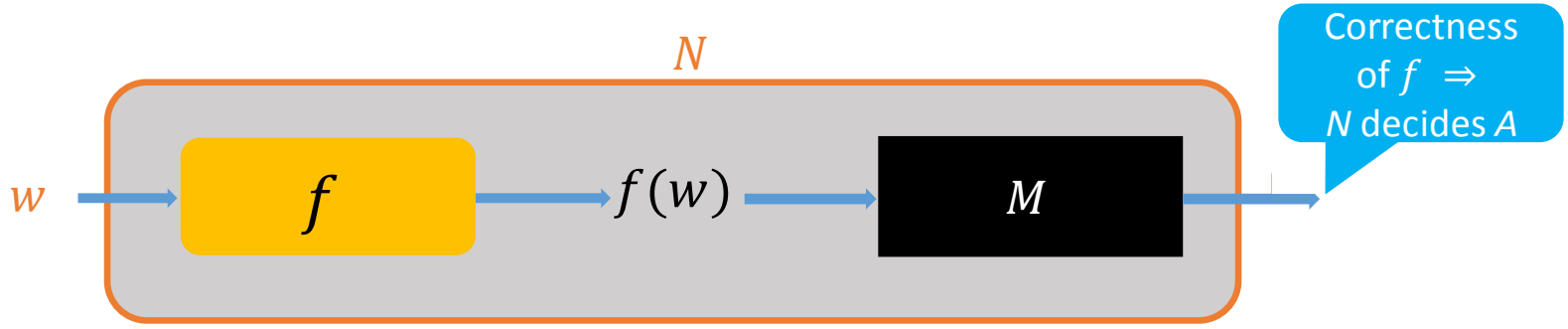
- $N =$ "On input w :
  1. Compute $f(w)$
  2. Run M on input $f(w)$ and output whatever M outputs"

Corollary: If $A \leq_m B$ and A is undecidable, then B is undecidable

Theorem: If $A \leq_m B$ and B is decidable, then A is decidable

Proof:

- Assume that M is a decided for B and f is a reduction from A to B
- Let N be a TM as follows:



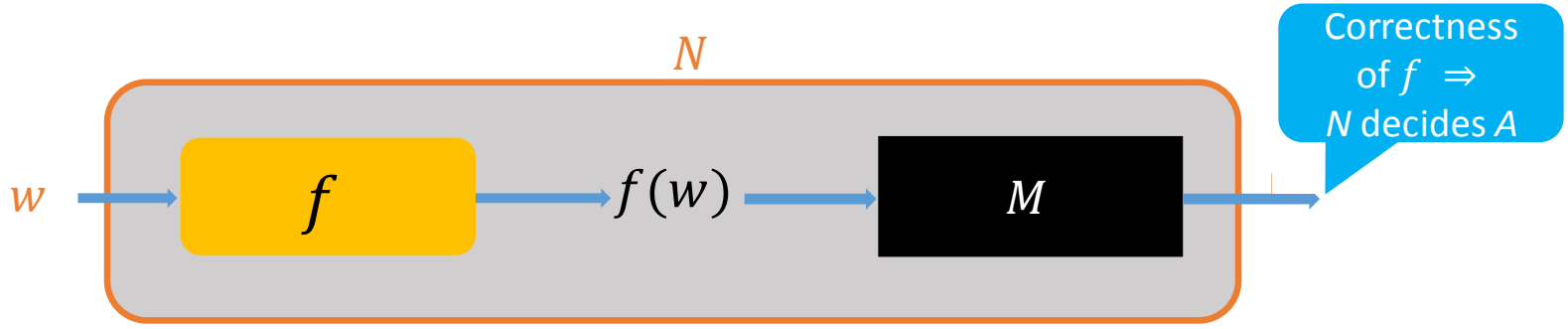
- N = "On input w :
  1. Compute $f(w)$
  2. Run M on input $f(w)$ and output whatever M outputs"

Computability of f ⇒ N is a decider

Theorem: If $A \leq_m B$ and B is decidable, then A is decidable

Proof:

- Assume that M is a decided for B and f is a reduction from A to B
- Let N be a TM as follows:



- N = "On input w :
  1. Compute $f(w)$
  2. Run M on input $f(w)$ and output whatever M outputs"

Computability of f ⇒ N is a decider

Corollary: If $A \leq_m B$ and A is undecidable, then B is undecidable

Examples of Reductions

$$HALT_{TM} = \{\langle M, w\rangle \mid M \text{ halts on } w\}$$
$$A_{TM} = \{\langle M, w\rangle \mid M \text{ accepts } w\}$$

$$HALT_{TM} = \{\langle M, w\rangle \mid M \text{ halts on } w\}$$
$$A_{TM} = \{\langle M, w\rangle \mid M \text{ accepts } w\}$$

Theorem: $A_{TM} \leq_m HALT_{TM}$

$$HALT_{TM} = \{\langle M, w\rangle \mid M \text{ halts on } w\}$$
$$A_{TM} = \{\langle M, w\rangle \mid M \text{ accepts } w\}$$

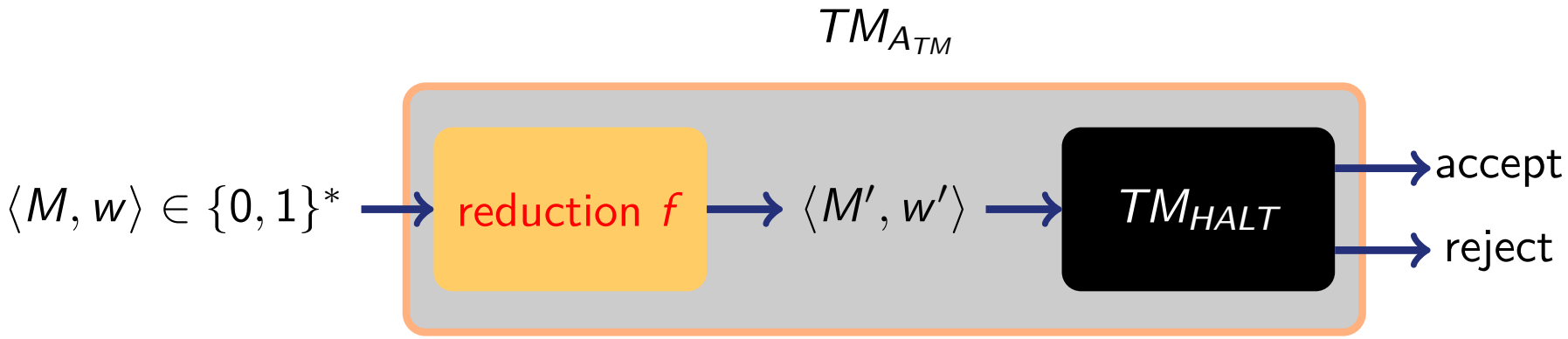
Theorem: $A_{TM} \leq_m HALT_{TM}$

Proof idea:

$$HALT_{TM} = \{\langle M, w\rangle \mid M \text{ halts on } w\}$$
$$A_{TM} = \{\langle M, w\rangle \mid M \text{ accepts } w\}$$

Theorem: $A_{TM} \leq_m HALT_{TM}$

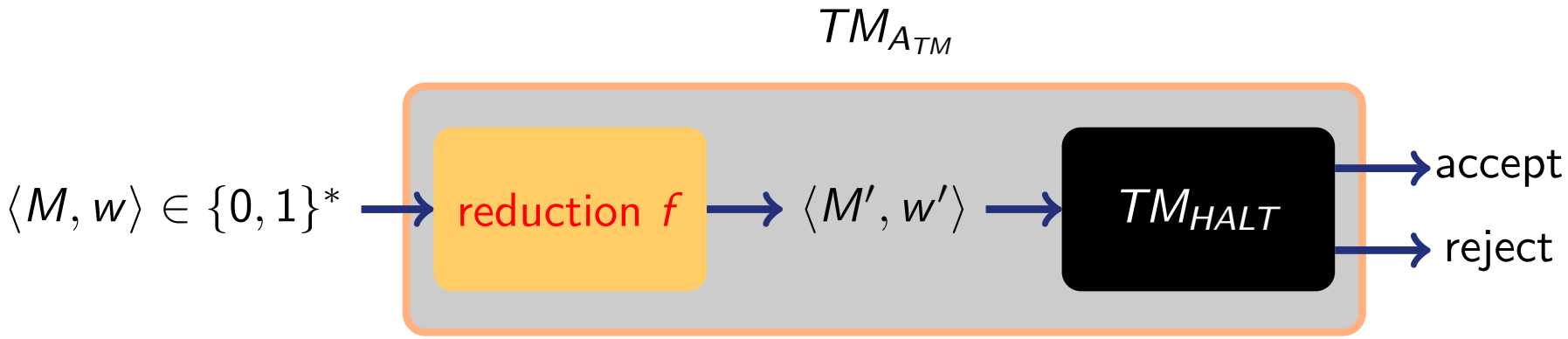
Proof idea:



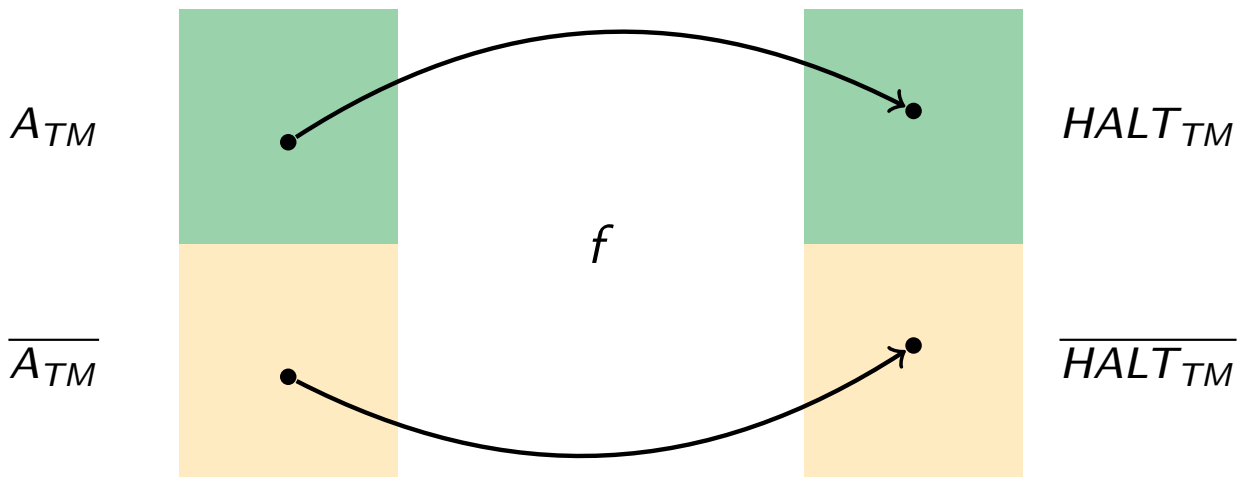
$$HALT_{TM} = \{\langle M, w\rangle \mid M \text{ halts on } w\}$$
$$A_{TM} = \{\langle M, w\rangle \mid M \text{ accepts } w\}$$

Theorem: $A_{TM} \leq_m HALT_{TM}$

Proof idea:



Define computable f such that



Theorem: $A_{TM} \leq_m HALT_{TM}$ ($\Rightarrow HALT_{TM}$ is undecidable)

Theorem: $A_{TM} \leq_m HALT_{TM}$ $(\Rightarrow HALT_{TM}$ is undecidable)

- Let us define a function f as follows

Theorem: $A_{TM} \leq_m HALT_{TM}$ $(\Rightarrow HALT_{TM}$ is undecidable)

- Let us define a function f as follows
- Given an input $x = \langle M, w \rangle$, return $f(x) = \langle M', w \rangle$, where
- M' ="On input y :
  1. Run M on w;
  2. **If M rejects w, enter infinite loop.** Otherwise, accept y"

Theorem: $A_{TM} \leq_m HALT_{TM}$ $(\Rightarrow HALT_{TM}$ is undecidable)

- Let us define a function f as follows
- Given an input $x = \langle M, w \rangle$, return $f(x) = \langle M', w \rangle$, where
- M' ="On input y :
  1. Run M on w;
  2. **If M rejects w, enter infinite loop.** Otherwise, accept y"
- Check that f is computable

Theorem: $A_{TM} \leq_m HALT_{TM}$ $(\Rightarrow HALT_{TM}$ is undecidable)

- Let us define a function f as follows
- Given an input $x = \langle M, w \rangle$, return $f(x) = \langle M', w \rangle$, where
- M' ="On input y :
  1. Run M on w;
  2. **If M rejects w, enter infinite loop.** Otherwise, accept y"
- Check that f is computable

f has to write down the code of M' only.
It does not run M'!
(M' might loop for some inputs.)

Theorem: $A_{TM} \leq_m HALT_{TM}$ $(\Rightarrow HALT_{TM}$ is undecidable)

- Let us define a function f as follows
- Given an input $x = \langle M, w \rangle$, return $f(x) = \langle M', w \rangle$, where
- M' ="On input y :
  1. Run M on w;
  2. **If M rejects w, enter infinite loop.** Otherwise, accept y"
- Check that f is computable

f has to write down the code of M' only.
It does not run M'!
(M' might loop for some inputs.)

- $\Rightarrow$ If $\langle M, w \rangle \in A_{TM}$ then M' halts on w.
  Thus, $\langle M', w \rangle \in HALT_{TM}$

Theorem: $A_{TM} \leq_m HALT_{TM}$ $(\Rightarrow HALT_{TM}$ is undecidable)

- Let us define a function f as follows
- Given an input $x = \langle M, w \rangle$, return $f(x) = \langle M', w \rangle$, where
- M' ="On input y :
  1. Run M on w;
  2. **If M rejects w, enter infinite loop.** Otherwise, accept y"
- Check that f is computable

f has to write down the code of M' only.
It does not run M'!
(M' might loop for some inputs.)

- $\Rightarrow$ If $\langle M, w \rangle \in A_{TM}$ then M' halts on w.
  Thus, $\langle M', w \rangle \in HALT_{TM}$
- $\Leftarrow$ If $\langle M, w \rangle \notin A_{TM}$ then either (1) will never halt or M rejects w and (2) will ensure no halting. Thus, $\langle M', w \rangle \notin HALT_{TM}$

$$REG_{TM} = \{\langle N \rangle \mid L(N) \text{ is a regular language}\}$$

$$REG_{TM} = \{\langle N \rangle \mid L(N) \text{ is a regular language}\}$$

Theorem: REG_{TM} is undecidable

$$REG_{TM} = \{\langle N \rangle \mid L(N) \text{ is a regular language}\}$$

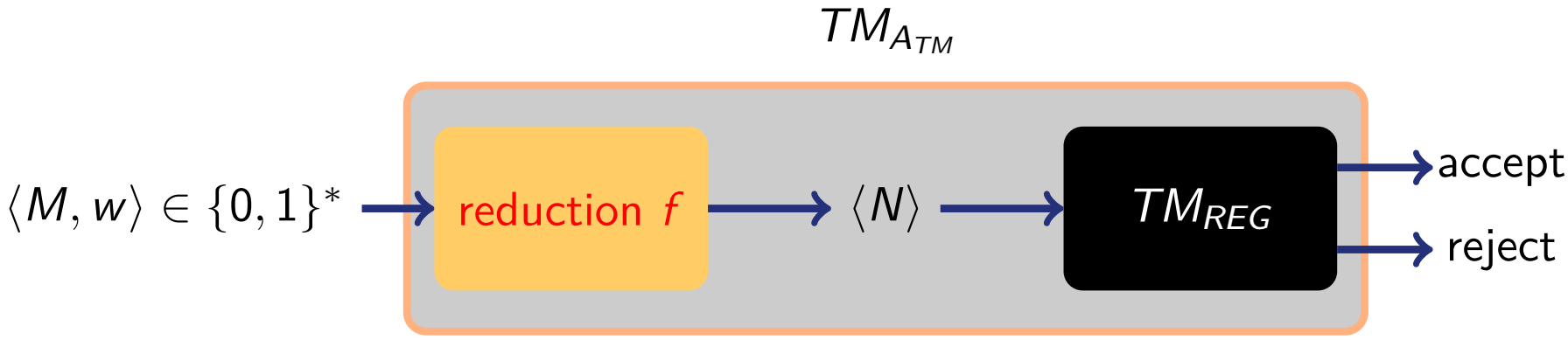
Theorem: REG_{TM} is undecidable

Proof idea: reduction from A_{TM}

$$REG_{TM} = \{\langle N \rangle \mid L(N) \text{ is a regular language}\}$$

Theorem: REG_{TM} is undecidable

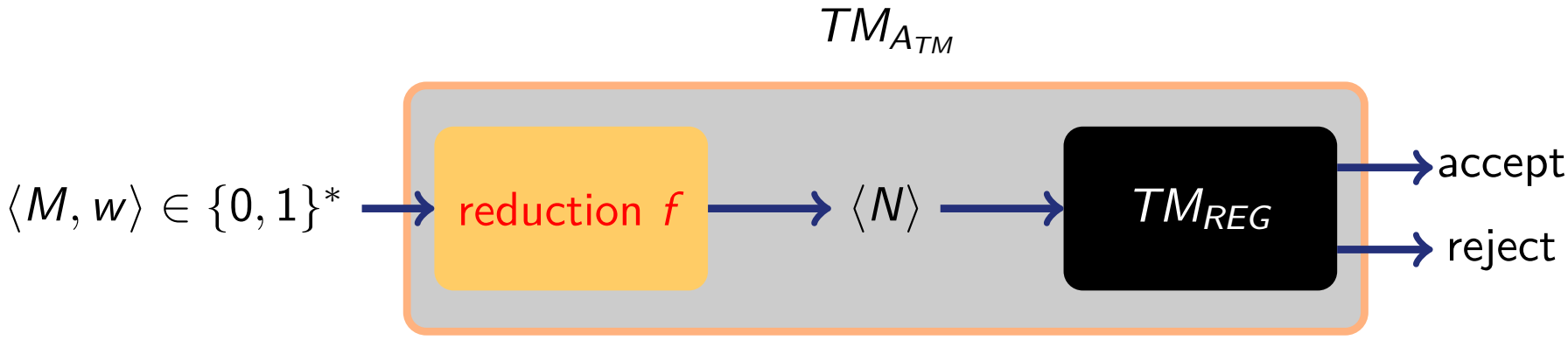
Proof idea: reduction from A_{TM}



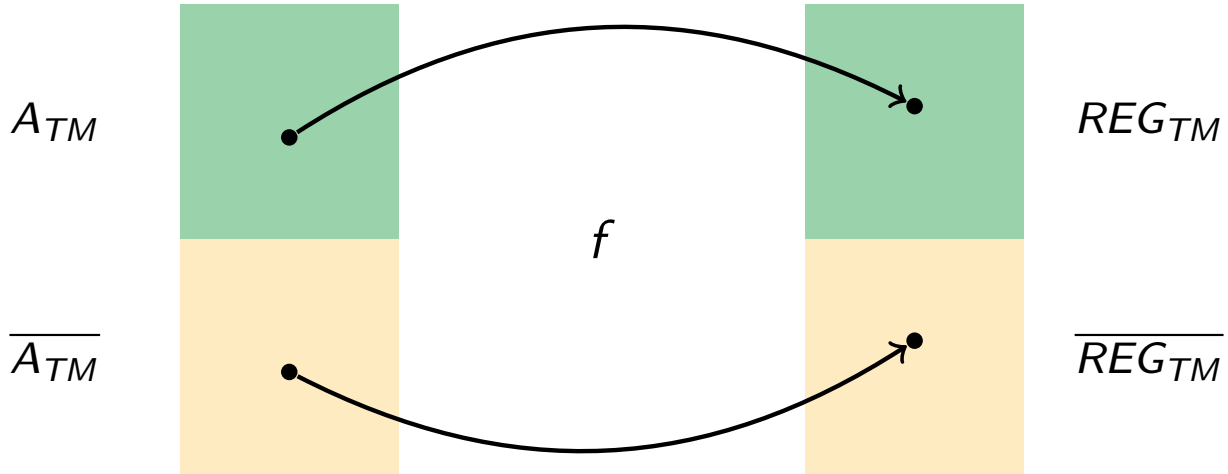
$$REG_{TM} = \{\langle N \rangle \mid L(N) \text{ is a regular language}\}$$

Theorem: REG_{TM} is undecidable

Proof idea: reduction from A_{TM}



Define computable f such that



Theorem: $A_{TM} \leq_m REG_{TM}$ $(\Rightarrow REG_{TM}$ is undecidable)

Theorem: $A_{TM} \leq_m REG_{TM}$ $(\Rightarrow REG_{TM}$ is undecidable)

- Let us define a function f as follows

Theorem: $A_{TM} \leq_m REG_{TM}$ $\quad (\Rightarrow REG_{TM}$ is undecidable)

- Let us define a function f as follows
- Given an input $x = \langle M, w \rangle$, return $f(x) = \langle M' \rangle$, where
- $M' =$"On input y :
  1. if $y \in B = \{0^n 1^n : n \geq 0\}$, then accept y
  2. Run M on w, and accept y iff M accepts w"

Theorem: $A_{TM} \leq_m REG_{TM}$ $(\Rightarrow REG_{TM}$ is undecidable)

- Let us define a function f as follows
- Given an input $x = \langle M, w \rangle$, return $f(x) = \langle M' \rangle$, where
- $M' =$"On input y :
  1. if $y \in B = \{0^n 1^n : n \geq 0\}$, then accept y
  2. Run M on w, and accept y iff M accepts w"
- Check that f is computable

Theorem: $A_{TM} \leq_m REG_{TM}$ $\quad (\Rightarrow REG_{TM}$ is undecidable)

- Let us define a function f as follows
- Given an input $x = \langle M, w \rangle$, return $f(x) = \langle M' \rangle$, where
- $M' =$"On input y :
  1. if $y \in B = \{0^n 1^n : n \geq 0\}$, then accept y
  2. Run M on w, and accept y iff M accepts w"
- Check that f is computable

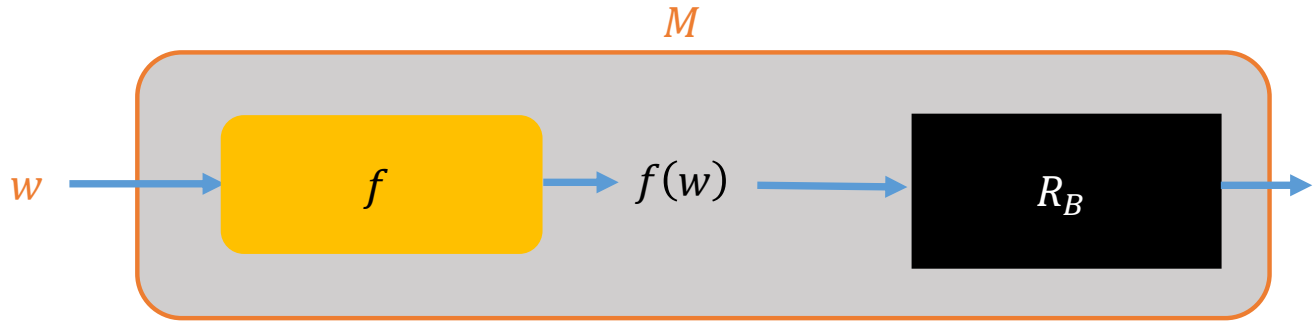
- $\Rightarrow$ If $\langle M, w \rangle \in A_{TM}$ then M' accepts all inputs.
  Thus, $L(M') = \{0, 1\}^*$ is regular — $\langle M' \rangle \in REG_{TM}$

Theorem: $A_{TM} \leq_m REG_{TM}$ $\quad (\Rightarrow REG_{TM}$ is undecidable)

- Let us define a function f as follows
- Given an input $x = \langle M, w \rangle$, return $f(x) = \langle M' \rangle$, where
- $M' =$"On input y :
  1. if $y \in B = \{0^n 1^n : n \geq 0\}$, then accept y
  2. Run M on w, and accept y iff M accepts w"
- Check that f is computable

- $\Rightarrow$ If $\langle M, w \rangle \in A_{TM}$ then M' accepts all inputs.
  Thus, $L(M') = \{0,1\}^*$ is regular — $\langle M' \rangle \in REG_{TM}$
- $\Leftarrow$ If $\langle M, w \rangle \notin A_{TM}$ then M does not accept w.
  Thus $L(M') = B$ is non-regular — $\langle M' \rangle \notin REG_{TM}$

Theorem: If $A \leq_m B$ and B is recognizable then A is recognizable



Definition of M

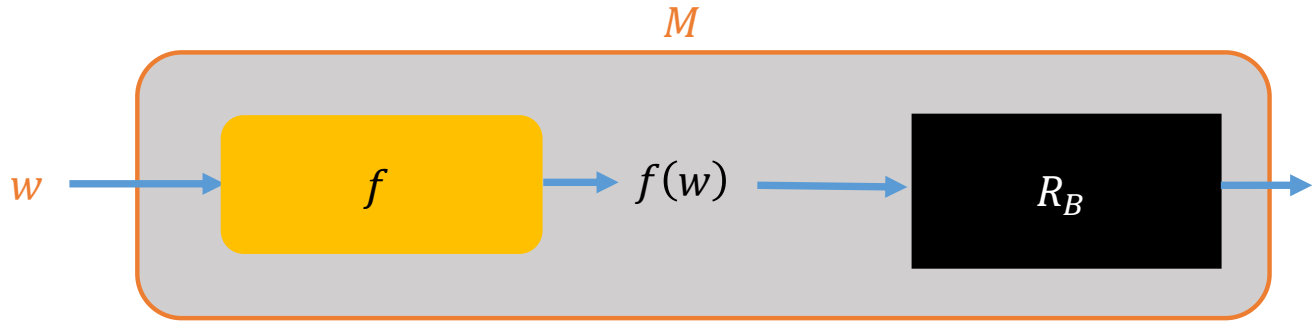
R_B is a recognizer for B

$A \leq_m B$

M accepts w $\Leftrightarrow$ R_B accepts $f(w)$ $\Leftrightarrow$ $f(w) \in B$ $\Leftrightarrow$ $w \in A$

$\Rightarrow$ **M is a recognizer for A**

Theorem: If $A \leq_m B$ and B is recognizable then A is recognizable



Definition of M

R_B is a recognizer for B

$A \leq_m B$

M accepts w $\Leftrightarrow$ R_B accepts $f(w)$ $\Leftrightarrow$ $f(w) \in B$ $\Leftrightarrow$ $w \in A$

$\Rightarrow$ **M is a recognizer for A**

Corollary: If $A \leq_m B$ and A is unrecognizable then B is unrecognizable

$$EQ_{TM} = \{\langle M_1, M_2 \rangle \mid M_1, M_2 \text{ are TMs s.t. } L(M_1) = L(M_2)\}$$

$$EQ_{TM} = \{\langle M_1, M_2 \rangle \mid M_1, M_2 \text{ are TMs s.t. } L(M_1) = L(M_2)\}$$

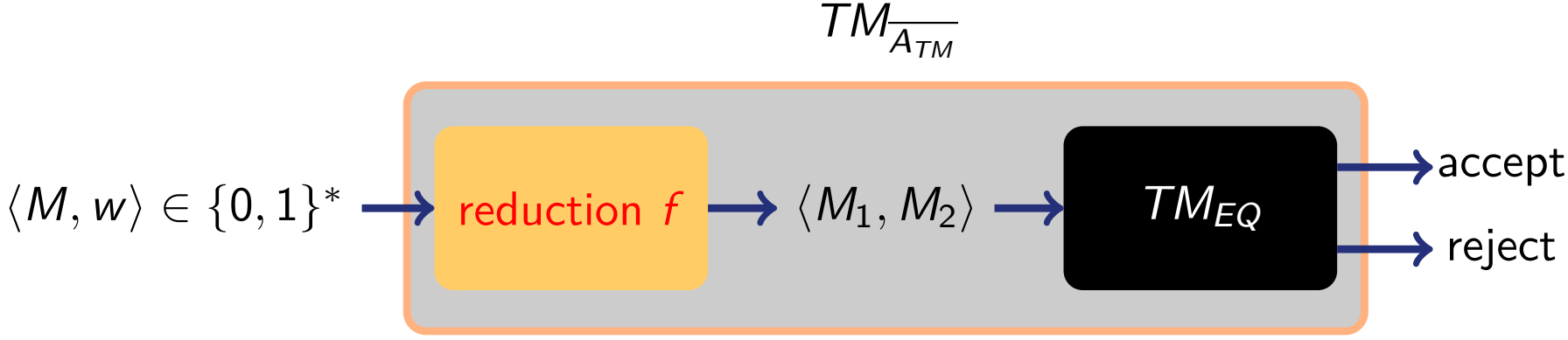
Theorem: EQ_{TM} is unrecognizable

Proof idea: reduction from $\overline{A_{TM}}$

$$EQ_{TM} = \{\langle M_1, M_2\rangle \mid M_1, M_2 \text{ are TMs s.t. } L(M_1) = L(M_2)\}$$

Theorem: EQ_{TM} is unrecognizable

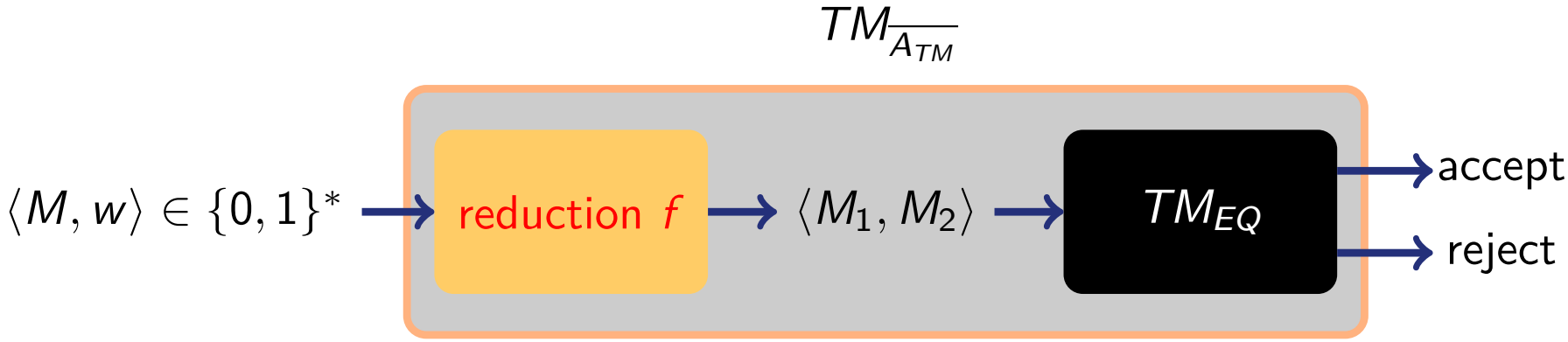
Proof idea: reduction from $\overline{A_{TM}}$



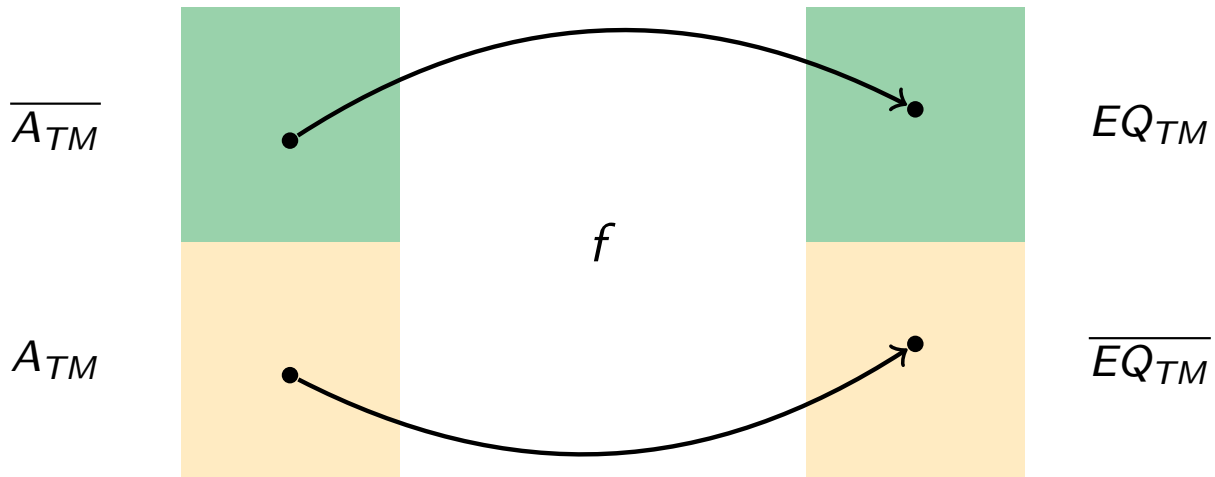
$$EQ_{TM} = \{\langle M_1, M_2\rangle \mid M_1, M_2 \text{ are TMs s.t. } L(M_1) = L(M_2)\}$$

Theorem: EQ_{TM} is unrecognizable

Proof idea: reduction from $\overline{A_{TM}}$



Define computable f such that



$$EQ_{TM} = \{\langle M_1, M_2 \rangle \mid M_1, M_2 \text{ are TMs s.t. } L(M_1) = L(M_2)\}$$

$$EQ_{TM} = \{\langle M_1, M_2 \rangle \mid M_1, M_2 \text{ are TMs s.t. } L(M_1) = L(M_2)\}$$

Theorem: $\overline{A_{TM}} \leq_m EQ_{TM}$ $\quad (\Rightarrow EQ_{TM}$ is unrecognizable)

- Let us define a function f as follows

$$EQ_{TM} = \{\langle M_1, M_2\rangle \mid M_1, M_2 \text{ are TMs s.t. } L(M_1) = L(M_2)\}$$

Theorem: $\overline{A_{TM}} \leq_m EQ_{TM}$ $\quad (\Rightarrow EQ_{TM}$ is unrecognizable)

- Let us define a function f as follows
- Given an input $x = \langle M, w\rangle$, return $f(x) = \langle M_1, M_2\rangle$, where
- $M_1 =$"On input y :
  1. Run M on w (ignore the input y)
  2. If M accepts then accept, else enter an infinite loop

$$EQ_{TM} = \{\langle M_1, M_2\rangle \mid M_1, M_2 \text{ are TMs s.t. } L(M_1) = L(M_2)\}$$

Theorem: $\overline{A_{TM}} \leq_m EQ_{TM}$ $\quad (\Rightarrow EQ_{TM}$ is unrecognizable)

- Let us define a function f as follows
- Given an input $x = \langle M, w\rangle$, return $f(x) = \langle M_1, M_2\rangle$, where
- M_1 ="On input y :
  1. Run M on w (ignore the input y)
  2. If M accepts then accept, else enter an infinite loop
- M_2 ="On input y :
  1. Reject y

$$EQ_{TM} = \{\langle M_1, M_2\rangle \mid M_1, M_2 \text{ are TMs s.t. } L(M_1) = L(M_2)\}$$

Theorem: $\overline{A_{TM}} \leq_m EQ_{TM}$ $\quad (\Rightarrow EQ_{TM}$ is unrecognizable)

- Let us define a function f as follows
- Given an input $x = \langle M, w\rangle$, return $f(x) = \langle M_1, M_2\rangle$, where
- M_1 ="On input y :
  1. Run M on w (ignore the input y)
  2. If M accepts then accept, else enter an infinite loop
- M_2 ="On input y :
  1. Reject y

- $\Rightarrow$ If $\langle M, w\rangle \in \overline{A_{TM}}$ then M_1 loops on all inputs.
  Thus, $L(M_1) = \emptyset = L(M_2)$ — $\langle M_1, M_2\rangle \in EQ_{TM}$

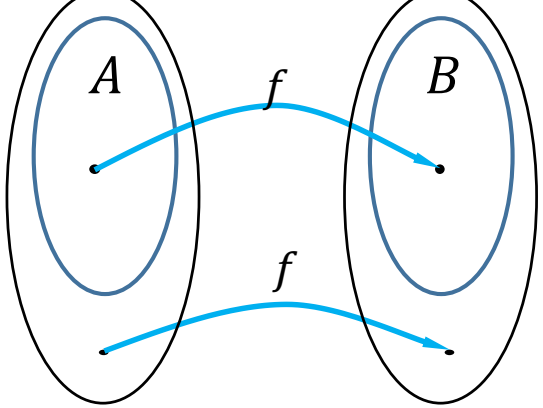
$$EQ_{TM} = \{\langle M_1, M_2\rangle \mid M_1, M_2 \text{ are TMs s.t. } L(M_1) = L(M_2)\}$$

Theorem: $\overline{A_{TM}} \leq_m EQ_{TM}$ $\quad (\Rightarrow EQ_{TM}$ is unrecognizable)

- Let us define a function f as follows
- Given an input $x = \langle M, w\rangle$, return $f(x) = \langle M_1, M_2\rangle$, where
- M_1 ="On input y :
  1. Run M on w (ignore the input y)
  2. If M accepts then accept, else enter an infinite loop
- M_2 ="On input y :
  1. Reject y

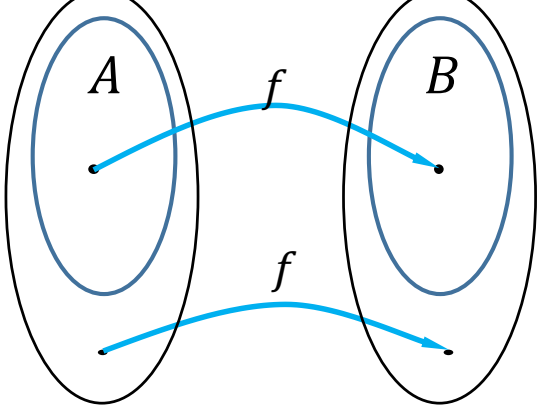
- $\Rightarrow$ If $\langle M, w\rangle \in \overline{A_{TM}}$ then M_1 loops on all inputs. Thus, $L(M_1) = \emptyset = L(M_2)$ — $\langle M_1, M_2\rangle \in EQ_{TM}$
- $\Leftarrow$ If $\langle M, w\rangle \notin \overline{A_{TM}}$ then M accepts w and hence M_1 accepts every string. Thus $L(M_1) = \Sigma^* \neq \emptyset = L(M_2)$ — $\langle M_1, M_2\rangle \notin EQ_{TM}$

Summary of Reductions



Definition: A function $f : \Sigma^* \to \Sigma^*$ is a *computable function* if some TM M, on every input w halts with just $f(w)$ on its tape

Summary of Reductions



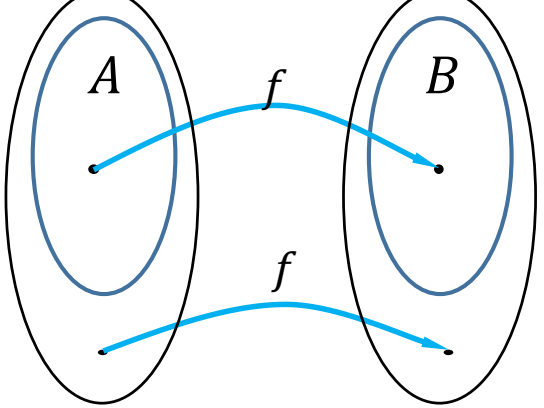
Definition: A function $f : \Sigma^* \to \Sigma^*$ is a *computable function* if some TM M, on every input w halts with just $f(w)$ on its tape

Definition: Language A is **mapping reducible** to language B, written $A \leq_m B$, if there is a computable function $f : \Sigma^* \to \Sigma^*$, such that for every $w \in \Sigma^*$:

$$w \in A \Leftrightarrow f(w) \in B$$

f is called a **reduction** of A to B

Summary of Reductions



Definition: A function $f : \Sigma^* \to \Sigma^*$ is a *computable function* if some TM M, on every input w halts with just $f(w)$ on its tape

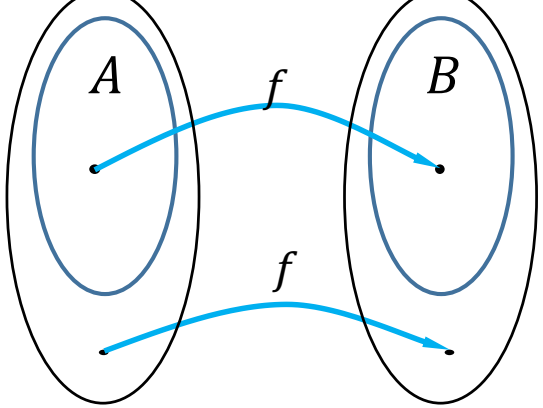
Definition: Language A is **mapping reducible** to language B, written $A \leq_m B$, if there is a computable function $f : \Sigma^* \to \Sigma^*$, such that for every $w \in \Sigma^*$:

$$w \in A \Leftrightarrow f(w) \in B$$

f is called a **reduction** of A to B

Theorem: If $A \leq_m B$ and B is decidable (recognizable), then A is decidable (recognizable)

Summary of Reductions



Definition: A function $f : \Sigma^* \to \Sigma^*$ is a *computable function* if some TM M, on every input w halts with just $f(w)$ on its tape

Definition: Language A is **mapping reducible** to language B, written $A \leq_m B$, if there is a computable function $f : \Sigma^* \to \Sigma^*$, such that for every $w \in \Sigma^*$:

$$w \in A \Leftrightarrow f(w) \in B$$

f is called a **reduction** of A to B

Theorem: If $A \leq_m B$ and B is decidable (recognizable), then A is decidable (recognizable)

Corollary: If $A \leq_m B$ and A is undecidable (unrecognizable) then B is undecidable (unrecognizable)

Some Exercises

True/False Questions

True/False Questions

1. A is regular $\Rightarrow$ A is decidable $\Rightarrow$ A is recognizable.

True/False Questions

1. A is regular $\Rightarrow$ A is decidable $\Rightarrow$ A is recognizable.
   True: every regular language is decidable, and every decidable language is recognizable.

True/False Questions

1. A is regular $\Rightarrow$ A is decidable $\Rightarrow$ A is recognizable.
   True: every regular language is decidable, and every decidable language is recognizable.
2. A and B are decidable $\Rightarrow$ $A \cup B$ is decidable.

True/False Questions

1. A is regular $\Rightarrow$ A is decidable $\Rightarrow$ A is recognizable.
   True: every regular language is decidable, and every decidable language is recognizable.
2. A and B are decidable $\Rightarrow$ $A \cup B$ is decidable.
   True: run the two deciders and accept iff at least one accepts.

True/False Questions

1. A is regular $\Rightarrow$ A is decidable $\Rightarrow$ A is recognizable.
   True: every regular language is decidable, and every decidable language is recognizable.
2. A and B are decidable $\Rightarrow$ $A \cup B$ is decidable.
   True: run the two deciders and accept iff at least one accepts.
3. A and B are unrecognizable $\Rightarrow$ $A \cup B$ is unrecognizable.

True/False Questions

1. A is regular $\Rightarrow$ A is decidable $\Rightarrow$ A is recognizable.
   True: every regular language is decidable, and every decidable language is recognizable.
2. A and B are decidable $\Rightarrow$ $A \cup B$ is decidable.
   True: run the two deciders and accept iff at least one accepts.
3. A and B are unrecognizable $\Rightarrow$ $A \cup B$ is unrecognizable.
   False: set $B := \bar{A}$ for any A s.t. $A, \bar{A}$ are unrecognizable.
   (e.g., set A to $AH = \{\langle M \rangle \mid M \text{ halts on every input}\}$)

True/False Questions

1. A is regular $\Rightarrow$ A is decidable $\Rightarrow$ A is recognizable.
   True: every regular language is decidable, and every decidable language is recognizable.
2. A and B are decidable $\Rightarrow$ $A \cup B$ is decidable.
   True: run the two deciders and accept iff at least one accepts.
3. A and B are unrecognizable $\Rightarrow$ $A \cup B$ is unrecognizable.
   False: set $B := \bar{A}$ for any A s.t. $A, \bar{A}$ are unrecognizable.
   (e.g., set A to $AH = \{\langle M \rangle \mid M \text{ halts on every input}\}$)
4. A is undecidable and recognizable $\Rightarrow$ $\overline{A}$ is unrecognizable.

True/False Questions

1. A is regular $\Rightarrow$ A is decidable $\Rightarrow$ A is recognizable.
   True: every regular language is decidable, and every decidable language is recognizable.
2. A and B are decidable $\Rightarrow$ $A \cup B$ is decidable.
   True: run the two deciders and accept iff at least one accepts.
3. A and B are unrecognizable $\Rightarrow$ $A \cup B$ is unrecognizable.
   False: set $B := \bar{A}$ for any A s.t. $A, \bar{A}$ are unrecognizable.
   (e.g., set A to $AH = \{\langle M \rangle \mid M \text{ halts on every input}\}$)
4. A is undecidable and recognizable $\Rightarrow$ $\overline{A}$ is unrecognizable.
   True: if both A and $\overline{A}$ were recognizable, then A would be decidable.

True/False Questions

1. A is regular $\Rightarrow$ A is decidable $\Rightarrow$ A is recognizable.
   True: every regular language is decidable, and every decidable language is recognizable.
2. A and B are decidable $\Rightarrow$ $A \cup B$ is decidable.
   True: run the two deciders and accept iff at least one accepts.
3. A and B are unrecognizable $\Rightarrow$ $A \cup B$ is unrecognizable.
   False: set $B := \bar{A}$ for any A s.t. $A, \bar{A}$ are unrecognizable.
   (e.g., set A to $AH = \{\langle M \rangle \mid M \text{ halts on every input}\}$)
4. A is undecidable and recognizable $\Rightarrow$ $\overline{A}$ is unrecognizable.
   True: if both A and $\overline{A}$ were recognizable, then A would be decidable.
5. $\overline{A}$ is decidable $\Rightarrow$ A is decidable.

True/False Questions

1. A is regular $\Rightarrow$ A is decidable $\Rightarrow$ A is recognizable.
   True: every regular language is decidable, and every decidable language is recognizable.
2. A and B are decidable $\Rightarrow$ $A \cup B$ is decidable.
   True: run the two deciders and accept iff at least one accepts.
3. A and B are unrecognizable $\Rightarrow$ $A \cup B$ is unrecognizable.
   False: set $B := \bar{A}$ for any A s.t. $A, \bar{A}$ are unrecognizable.
   (e.g., set A to $AH = \{\langle M \rangle \mid M \text{ halts on every input}\}$)
4. A is undecidable and recognizable $\Rightarrow$ $\overline{A}$ is unrecognizable.
   True: if both A and $\overline{A}$ were recognizable, then A would be decidable.
5. $\overline{A}$ is decidable $\Rightarrow$ A is decidable.
   True: decidability is closed under complement.

True/False Questions

1. A is regular $\Rightarrow$ A is decidable $\Rightarrow$ A is recognizable.
   True: every regular language is decidable, and every decidable language is recognizable.
2. A and B are decidable $\Rightarrow$ $A \cup B$ is decidable.
   True: run the two deciders and accept iff at least one accepts.
3. A and B are unrecognizable $\Rightarrow$ $A \cup B$ is unrecognizable.
   False: set $B := \bar{A}$ for any A s.t. $A, \bar{A}$ are unrecognizable.
   (e.g., set A to $AH = \{\langle M \rangle \mid M \text{ halts on every input}\}$)
4. A is undecidable and recognizable $\Rightarrow$ $\overline{A}$ is unrecognizable.
   True: if both A and $\overline{A}$ were recognizable, then A would be decidable.
5. $\overline{A}$ is decidable $\Rightarrow$ A is decidable.
   True: decidability is closed under complement.
6. $\{0^n 1^n : n \geq 0\}$ is decidable.

True/False Questions

1. A is regular $\Rightarrow$ A is decidable $\Rightarrow$ A is recognizable.
   True: every regular language is decidable, and every decidable language is recognizable.

2. A and B are decidable $\Rightarrow$ $A \cup B$ is decidable.
   True: run the two deciders and accept iff at least one accepts.

3. A and B are unrecognizable $\Rightarrow$ $A \cup B$ is unrecognizable.
   False: set $B := \bar{A}$ for any A s.t. $A, \bar{A}$ are unrecognizable.
   (e.g., set A to $AH = \{\langle M \rangle \mid M \text{ halts on every input}\}$)

4. A is undecidable and recognizable $\Rightarrow$ $\overline{A}$ is unrecognizable.
   True: if both A and $\overline{A}$ were recognizable, then A would be decidable.

5. $\overline{A}$ is decidable $\Rightarrow$ A is decidable.
   True: decidability is closed under complement.

6. $\{0^n1^n : n \geq 0\}$ is decidable.
   True: $\{0^n1^n : n \geq 0\}$ is a standard decidable language.

True or false: A is decidable and $A \leq_m B \Rightarrow B$ is decidable

False.

True or false: A is decidable and $A \leq_m B \Rightarrow B$ is decidable

False.

Claim: A is decidable $\Rightarrow A \leq_m B$ for every $B \neq \emptyset, \Sigma^*$.

True or false: A is decidable and $A \leq_m B \Rightarrow B$ is decidable

False.

Claim: A is decidable $\Rightarrow A \leq_m B$ for every $B \neq \emptyset, \Sigma^*$.

- Fix strings $y \in B$ and $y' \in \overline{B}$.
- A is decidable so this function is computable

$$f(x) := \begin{cases} y & \text{if } x \in A \\ y' & \text{if } x \notin A \end{cases}.$$

- Then $x \in A \Leftrightarrow f(x) \in B$, so $A \leq_m B$.

True or false: A is decidable and $A \leq_m B \Rightarrow B$ is decidable

False.

Claim: A is decidable $\Rightarrow A \leq_m B$ for every $B \neq \emptyset, \Sigma^*$.

- Fix strings $y \in B$ and $y' \in \overline{B}$.
- A is decidable so this function is computable

$$f(x) := \begin{cases} y & \text{if } x \in A \\ y' & \text{if } x \notin A \end{cases}.$$

- Then $x \in A \Leftrightarrow f(x) \in B$, so $A \leq_m B$.

Take $A :=$ "any decidable nontrivial language" and $B := HALT_{TM}$.

True or false: This language is recognizable:

$$\overline{HALT_{TM}} := \{\langle M, w\rangle \mid M \text{ is a TM that does not halt on input } w\} \ .$$

True or false: This language is recognizable:

$$\overline{HALT_{TM}} := \{\langle M, w\rangle \mid M \text{ is a TM that does not halt on input } w\} \ .$$

False.

True or false: This language is recognizable:

$$\overline{HALT_{TM}} := \{\langle M, w\rangle \mid M \text{ is a TM that does not halt on input } w\} \ .$$

False.

$HALT_{TM}$ is recognizable so if $\overline{HALT_{TM}}$ were recognizable, we would deduce that $HALT_{TM}$ is decidable, a contradiction.

True or false: This language is recognizable:

$$\overline{HALT_{TM}} := \{\langle M, w\rangle \mid M \text{ is a TM that does not halt on input } w\} \ .$$

False.

$HALT_{TM}$ is recognizable so if $\overline{HALT_{TM}}$ were recognizable, we would deduce that $HALT_{TM}$ is decidable, a contradiction.

Notes:

True or false: This language is recognizable:

$$\overline{HALT_{TM}} := \{\langle M, w\rangle \mid M \text{ is a TM that does not halt on input } w\} \ .$$

False.

$HALT_{TM}$ is recognizable so if $\overline{HALT_{TM}}$ were recognizable, we would deduce that $HALT_{TM}$ is decidable, a contradiction.

Notes:

- A is decidable $\Rightarrow$ $HALT_{TM} \not\leq_m A$
  (for otherwise $HALT_{TM}$ would be decidable).

True or false: This language is recognizable:

$$\overline{HALT_{TM}} := \{\langle M, w\rangle \mid M \text{ is a TM that does not halt on input } w\} \ .$$

False.

$HALT_{TM}$ is recognizable so if $\overline{HALT_{TM}}$ were recognizable, we would deduce that $HALT_{TM}$ is decidable, a contradiction.

Notes:

- A is decidable $\Rightarrow$ $HALT_{TM} \not\leq_m A$
  (for otherwise $HALT_{TM}$ would be decidable).
- $\overline{HALT_{TM}} \not\leq_m HALT_{TM}$
  ($HALT_{TM}$ is recognizable and if $\overline{HALT_{TM}} \leq_m HALT_{TM}$ then $\overline{HALT_{TM}}$ would be recognizable, a contradiction)

Exercise: Show that

$$\big(B \text{ undecidable and } B \leq_m \overline{B}\big) \Rightarrow \big(B \text{ and } \overline{B} \text{ are unrecognizable}\big).$$

Exercise: Show that

$$\big(B \text{ undecidable and } B \leq_m \overline{B}\big) \Rightarrow \big(B \text{ and } \overline{B} \text{ are unrecognizable}\big).$$

We prove the contrapositive:

$$\big(B \text{ recognizable or } \overline{B} \text{ recognizable}\big) \Rightarrow \big(B \text{ decidable or } B \not\leq_m \overline{B}\big).$$

Exercise: Show that

$$\big(B \text{ undecidable and } B \leq_m \overline{B}\big) \Rightarrow \big(B \text{ and } \overline{B} \text{ are unrecognizable}\big).$$

We prove the contrapositive:

$$\big(B \text{ recognizable or } \overline{B} \text{ recognizable}\big) \Rightarrow \big(B \text{ decidable or } B \not\leq_m \overline{B}\big).$$

For mapping reductions, $A \leq_m B \iff \overline{A} \leq_m \overline{B}$.

Exercise: Show that

$$\big(B \text{ undecidable and } B \leq_m \overline{B}\big) \Rightarrow \big(B \text{ and } \overline{B} \text{ are unrecognizable}\big).$$

We prove the contrapositive:

$$\big(B \text{ recognizable or } \overline{B} \text{ recognizable}\big) \Rightarrow \big(B \text{ decidable or } B \not\leq_m \overline{B}\big).$$

For mapping reductions, $A \leq_m B \iff \overline{A} \leq_m \overline{B}$.
Assume that B is recognizable and $B \leq_m \overline{B}$.

Exercise: Show that

$$\big(B \text{ undecidable and } B \leq_m \overline{B}\big) \Rightarrow \big(B \text{ and } \overline{B} \text{ are unrecognizable}\big).$$

We prove the contrapositive:

$$\big(B \text{ recognizable or } \overline{B} \text{ recognizable}\big) \Rightarrow \big(B \text{ decidable or } B \not\leq_m \overline{B}\big).$$

For mapping reductions, $A \leq_m B \iff \overline{A} \leq_m \overline{B}$.
Assume that B is recognizable and $B \leq_m \overline{B}$.
By the complement rule, $\overline{B} \leq_m B$.

Exercise: Show that

$$\big(B \text{ undecidable and } B \leq_m \overline{B}\big) \Rightarrow \big(B \text{ and } \overline{B} \text{ are unrecognizable}\big).$$

We prove the contrapositive:

$$\big(B \text{ recognizable or } \overline{B} \text{ recognizable}\big) \Rightarrow \big(B \text{ decidable or } B \not\leq_m \overline{B}\big).$$

For mapping reductions, $A \leq_m B \iff \overline{A} \leq_m \overline{B}$.
Assume that B is recognizable and $B \leq_m \overline{B}$.
By the complement rule, $\overline{B} \leq_m B$.
Since B is recognizable and $\overline{B} \leq_m B$, so $\overline{B}$ is recognizable.

Exercise: Show that

$$\big(B \text{ undecidable and } B \leq_m \overline{B}\big) \Rightarrow \big(B \text{ and } \overline{B} \text{ are unrecognizable}\big).$$

We prove the contrapositive:

$$\big(B \text{ recognizable or } \overline{B} \text{ recognizable}\big) \Rightarrow \big(B \text{ decidable or } B \not\leq_m \overline{B}\big).$$

For mapping reductions, $A \leq_m B \iff \overline{A} \leq_m \overline{B}$.
Assume that B is recognizable and $B \leq_m \overline{B}$.
By the complement rule, $\overline{B} \leq_m B$.
Since B is recognizable and $\overline{B} \leq_m B$, so $\overline{B}$ is recognizable.
Therefore both B and $\overline{B}$ are recognizable, so B is decidable.

Exercise: Show that

$$\big(B \text{ undecidable and } B \leq_m \overline{B}\big) \Rightarrow \big(B \text{ and } \overline{B} \text{ are unrecognizable}\big).$$

We prove the contrapositive:

$$\big(B \text{ recognizable or } \overline{B} \text{ recognizable}\big) \Rightarrow \big(B \text{ decidable or } B \not\leq_m \overline{B}\big).$$

For mapping reductions, $A \leq_m B \iff \overline{A} \leq_m \overline{B}$.
Assume that B is recognizable and $B \leq_m \overline{B}$.
By the complement rule, $\overline{B} \leq_m B$.
Since B is recognizable and $\overline{B} \leq_m B$, so $\overline{B}$ is recognizable.
Therefore both B and $\overline{B}$ are recognizable, so B is decidable.

The same argument works if we start with $\overline{B}$ recognizable instead.

A language built from $HALT_{TM}$ and $\overline{HALT_{TM}}$:

$$L^* := \{\langle x, y\rangle \mid x \in HALT_{TM} \text{ and } y \in \overline{HALT_{TM}}\}.$$

A language built from $HALT_{TM}$ and $\overline{HALT_{TM}}$:

$$L^* := \{\langle x, y\rangle \mid x \in HALT_{TM} \text{ and } y \in \overline{HALT_{TM}}\}.$$

Claim 1: $HALT_{TM} \leq_m L^*$

A language built from $HALT_{TM}$ and $\overline{HALT_{TM}}$:

$$L^* := \{\langle x, y\rangle \mid x \in HALT_{TM} \text{ and } y \in \overline{HALT_{TM}}\}.$$

Claim 1: $HALT_{TM} \leq_m L^*$

Fix any $y_0 \in \overline{HALT_{TM}}$ (eg a machine-input pair that loops forever).

A language built from $HALT_{TM}$ and $\overline{HALT_{TM}}$:

$$L^* := \{\langle x, y\rangle \mid x \in HALT_{TM} \text{ and } y \in \overline{HALT_{TM}}\}.$$

Claim 1: $HALT_{TM} \leq_m L^*$

Fix any $y_0 \in \overline{HALT_{TM}}$ (eg a machine-input pair that loops forever).
Define $f(x) := \langle x, y_0\rangle$.

A language built from $HALT_{TM}$ and $\overline{HALT_{TM}}$:

$$L^* := \{\langle x, y\rangle \mid x \in HALT_{TM} \text{ and } y \in \overline{HALT_{TM}}\}.$$

Claim 1: $HALT_{TM} \leq_m L^*$

Fix any $y_0 \in \overline{HALT_{TM}}$ (eg a machine-input pair that loops forever).
Define $f(x) := \langle x, y_0\rangle$.
Then

$$x \in HALT_{TM} \iff \langle x, y_0\rangle \in L^*.$$

A language built from $HALT_{TM}$ and $\overline{HALT_{TM}}$:

$$L^* := \{\langle x, y\rangle \mid x \in HALT_{TM} \text{ and } y \in \overline{HALT_{TM}}\}.$$

Claim 1: $HALT_{TM} \leq_m L^*$

Fix any $y_0 \in \overline{HALT_{TM}}$ (eg a machine-input pair that loops forever).
Define $f(x) := \langle x, y_0\rangle$.
Then

$$x \in HALT_{TM} \iff \langle x, y_0\rangle \in L^*.$$

Claim 2: $\overline{HALT_{TM}} \leq_m L^*$

A language built from $HALT_{TM}$ and $\overline{HALT_{TM}}$:

$$L^* := \{\langle x, y\rangle \mid x \in HALT_{TM} \text{ and } y \in \overline{HALT_{TM}}\}.$$

Claim 1: $HALT_{TM} \leq_m L^*$

Fix any $y_0 \in \overline{HALT_{TM}}$ (eg a machine-input pair that loops forever).
Define $f(x) := \langle x, y_0\rangle$.
Then

$$x \in HALT_{TM} \iff \langle x, y_0\rangle \in L^*.$$

Claim 2: $\overline{HALT_{TM}} \leq_m L^*$

Fix any $x_0 \in HALT_{TM}$.

A language built from $HALT_{TM}$ and $\overline{HALT_{TM}}$:

$$L^* := \{\langle x, y\rangle \mid x \in HALT_{TM} \text{ and } y \in \overline{HALT_{TM}}\}.$$

Claim 1: $HALT_{TM} \leq_m L^*$

Fix any $y_0 \in \overline{HALT_{TM}}$ (eg a machine-input pair that loops forever).
Define $f(x) := \langle x, y_0\rangle$.
Then

$$x \in HALT_{TM} \iff \langle x, y_0\rangle \in L^*.$$

Claim 2: $\overline{HALT_{TM}} \leq_m L^*$

Fix any $x_0 \in HALT_{TM}$.
Define $g(y) := \langle x_0, y\rangle$.

A language built from $HALT_{TM}$ and $\overline{HALT_{TM}}$:

$$L^* := \{\langle x, y\rangle \mid x \in HALT_{TM} \text{ and } y \in \overline{HALT_{TM}}\}.$$

Claim 1: $HALT_{TM} \leq_m L^*$

Fix any $y_0 \in \overline{HALT_{TM}}$ (eg a machine-input pair that loops forever).
Define $f(x) := \langle x, y_0\rangle$.
Then

$$x \in HALT_{TM} \iff \langle x, y_0\rangle \in L^*.$$

Claim 2: $\overline{HALT_{TM}} \leq_m L^*$

Fix any $x_0 \in HALT_{TM}$.
Define $g(y) := \langle x_0, y\rangle$.
Then

$$y \in \overline{HALT_{TM}} \iff \langle x_0, y\rangle \in L^*.$$

A language built from $HALT_{TM}$ and $\overline{HALT_{TM}}$:

$$L^* := \{\langle x, y\rangle \mid x \in HALT_{TM} \text{ and } y \in \overline{HALT_{TM}}\}.$$

Claim 1: $HALT_{TM} \leq_m L^*$

Fix any $y_0 \in \overline{HALT_{TM}}$ (eg a machine-input pair that loops forever).
Define $f(x) := \langle x, y_0\rangle$.
Then

$$x \in HALT_{TM} \iff \langle x, y_0\rangle \in L^*.$$

Claim 2: $\overline{HALT_{TM}} \leq_m L^*$

Fix any $x_0 \in HALT_{TM}$.
Define $g(y) := \langle x_0, y\rangle$.
Then

$$y \in \overline{HALT_{TM}} \iff \langle x_0, y\rangle \in L^*.$$

$\overline{HALT_{TM}}$ is unrecognizable and $\overline{HALT_{TM}} \leq_m L^*$, so L^* is unrecognizable.
In particular, L^* is undecidable.

From not rejecting to all halting

Consider the languages

$$NR := \{\langle M, w\rangle \mid M \text{ does not reject input } w\} \ ,$$
$$AH := \{\langle M\rangle \mid M \text{ halts on every input}\} \ .$$

From not rejecting to all halting

Consider the languages

$$NR := \{\langle M, w\rangle \mid M \text{ does not reject input } w\} \ ,$$
$$AH := \{\langle M\rangle \mid M \text{ halts on every input}\} \ .$$

Exercise: show that AH is unrecognizable, using the fact that NR is unrecognizable.

From not rejecting to all halting

Consider the languages

$$NR := \{\langle M, w\rangle \mid M \text{ does not reject input } w\} \ ,$$
$$AH := \{\langle M\rangle \mid M \text{ halts on every input}\} \ .$$

Exercise: show that AH is unrecognizable, using the fact that NR is unrecognizable.

Strategy: show that $NR \leq_m AH$.

From not rejecting to all halting

Consider the languages

$$NR := \{\langle M, w\rangle \mid M \text{ does not reject input } w\} \ ,$$
$$AH := \{\langle M\rangle \mid M \text{ halts on every input}\} \ .$$

Exercise: show that AH is unrecognizable, using the fact that NR is unrecognizable.

Strategy: show that $NR \leq_m AH$.

We seek f such that $f(\langle M, w\rangle) = \langle M'\rangle$ satisfies

$$\langle M, w\rangle \in NR \iff \langle M'\rangle \in AH \ .$$

From not rejecting to all halting

Consider the languages

$$NR := \{\langle M, w\rangle \mid M \text{ does not reject input } w\} \ ,$$
$$AH := \{\langle M\rangle \mid M \text{ halts on every input}\} \ .$$

Exercise: show that AH is unrecognizable, using the fact that NR is unrecognizable.

Strategy: show that $NR \leq_m AH$.

We seek f such that $f(\langle M, w\rangle) = \langle M'\rangle$ satisfies

$$\langle M, w\rangle \in NR \iff \langle M'\rangle \in AH \ .$$

Equivalently:

From not rejecting to all halting

Consider the languages

$$NR := \{\langle M, w\rangle \mid M \text{ does not reject input } w\} \ ,$$
$$AH := \{\langle M\rangle \mid M \text{ halts on every input}\} \ .$$

Exercise: show that AH is unrecognizable, using the fact that NR is unrecognizable.

Strategy: show that $NR \leq_m AH$.

We seek f such that $f(\langle M, w\rangle) = \langle M'\rangle$ satisfies

$$\langle M, w\rangle \in NR \iff \langle M'\rangle \in AH \ .$$

Equivalently:

- if M does not reject w (M accepts w or loops on w), then M' halts on every input;

From not rejecting to all halting

Consider the languages

$$NR := \{\langle M, w\rangle \mid M \text{ does not reject input } w\} \ ,$$
$$AH := \{\langle M\rangle \mid M \text{ halts on every input}\} \ .$$

Exercise: show that AH is unrecognizable, using the fact that NR is unrecognizable.

Strategy: show that $NR \leq_m AH$.

We seek f such that $f(\langle M, w\rangle) = \langle M'\rangle$ satisfies

$$\langle M, w\rangle \in NR \iff \langle M'\rangle \in AH \ .$$

Equivalently:

- if M does not reject w (M accepts w or loops on w), then M' halts on every input;
- if M rejects w, then M' fails to halt on some input.

$M' =$ "On input $i \in \mathbb{N}$:

$M' =$ "On input $i \in \mathbb{N}$:

1. Simulate M on input w for exactly i steps.

M' = "On input $i \in \mathbb{N}$:

1. Simulate M on input w for exactly i steps.
2. If within those i steps M rejects w, then enter an infinite loop.

M' = "On input $i \in \mathbb{N}$:

1. Simulate M on input w for exactly i steps.
2. If within those i steps M rejects w, then enter an infinite loop.
3. If within those i steps M accepts w, then halt.

M' = "On input $i \in \mathbb{N}$:

1. Simulate M on input w for exactly i steps.
2. If within those i steps M rejects w, then enter an infinite loop.
3. If within those i steps M accepts w, then halt.
4. If M has not halted within those i steps, then halt."

M' = "On input $i \in \mathbb{N}$:

1. Simulate M on input w for exactly i steps.
2. If within those i steps M rejects w, then enter an infinite loop.
3. If within those i steps M accepts w, then halt.
4. If M has not halted within those i steps, then halt."

This construction is computable, because from $\langle M, w \rangle$ we can derive a TM M' implementing the bounded simulation above.

M' = "On input $i \in \mathbb{N}$:

1. Simulate M on input w for exactly i steps.
2. If within those i steps M rejects w, then enter an infinite loop.
3. If within those i steps M accepts w, then halt.
4. If M has not halted within those i steps, then halt."

This construction is computable, because from $\langle M, w \rangle$ we can derive a TM M' implementing the bounded simulation above.

- Suppose $\langle M, w \rangle \in NR$ (M does not reject w).

M' = "On input $i \in \mathbb{N}$:

1. Simulate M on input w for exactly i steps.
2. If within those i steps M rejects w, then enter an infinite loop.
3. If within those i steps M accepts w, then halt.
4. If M has not halted within those i steps, then halt."

This construction is computable, because from $\langle M, w \rangle$ we can derive a TM M' implementing the bounded simulation above.

- Suppose $\langle M, w \rangle \in NR$ (M does not reject w).
  - If M accepts w after t steps, then $\forall\, i < t$ $M'(i)$ halts by step (4), and $\forall\, i \geq t$, $M'(i)$ halts by step (3).

M' = "On input $i \in \mathbb{N}$:

1. Simulate M on input w for exactly i steps.
2. If within those i steps M rejects w, then enter an infinite loop.
3. If within those i steps M accepts w, then halt.
4. If M has not halted within those i steps, then halt."

This construction is computable, because from $\langle M, w \rangle$ we can derive a TM M' implementing the bounded simulation above.

- Suppose $\langle M, w \rangle \in NR$ (M does not reject w).
  - If M accepts w after t steps, then $\forall\, i < t$ $M'(i)$ halts by step (4), and $\forall\, i \geq t$, $M'(i)$ halts by step (3).
  - If M never halts on w, then for every input i, machine M' halts by step (4).

M' = "On input $i \in \mathbb{N}$:

1. Simulate M on input w for exactly i steps.
2. If within those i steps M rejects w, then enter an infinite loop.
3. If within those i steps M accepts w, then halt.
4. If M has not halted within those i steps, then halt."

This construction is computable, because from $\langle M, w \rangle$ we can derive a TM M' implementing the bounded simulation above.

- Suppose $\langle M, w \rangle \in NR$ (M does not reject w).
  - If M accepts w after t steps, then $\forall\, i < t$ $M'(i)$ halts by step (4), and $\forall\, i \geq t$, $M'(i)$ halts by step (3).
  - If M never halts on w, then for every input i, machine M' halts by step (4).

  Hence M' halts on every input, so $\langle M' \rangle \in AH$.

M' = "On input $i \in \mathbb{N}$:

1. Simulate M on input w for exactly i steps.
2. If within those i steps M rejects w, then enter an infinite loop.
3. If within those i steps M accepts w, then halt.
4. If M has not halted within those i steps, then halt."

This construction is computable, because from $\langle M, w \rangle$ we can derive a TM M' implementing the bounded simulation above.

- Suppose $\langle M, w \rangle \in NR$ (M does not reject w).
  - If M accepts w after t steps, then $\forall\, i < t$ $M'(i)$ halts by step (4), and $\forall\, i \geq t$, $M'(i)$ halts by step (3).
  - If M never halts on w, then for every input i, machine M' halts by step (4).

  Hence M' halts on every input, so $\langle M' \rangle \in AH$.
- Suppose $\langle M, w \rangle \notin NR$; so M rejects w after some number t of steps. For every input $i \geq t$, step (2) makes M' loop forever. Hence M' does not halt on every input, so $\langle M' \rangle \notin AH$.